

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ



ĐỒ ÁN MÔN HỌC

**THIẾT KẾ CPU RISC-V RV32I ĐA CHU
KỲ THEO KIẾN TRÚC PIPELINE
TRÊN KIT FPGA PYNQ-Z2**

GVHD: TS. Võ Quế Sơn

SVTH: Phạm Vũ Hoàng Phúc

MSSV: 2312715

Thành phố Hồ Chí Minh, tháng 5 năm 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến thầy Võ Quế Sơn đã nhiệt tình giúp đỡ và chỉ dẫn em trong suốt quá trình thực hiện đề tài. Nhờ sự hướng dẫn tận tình và những góp ý quý báu của thầy mà em đã hoàn thành tốt được đề án. Em xin gửi lời cảm ơn đến quý thầy cô đã dạy dỗ và cung cấp kiến thức có liên quan để em có thể thực hiện được đề tài này một cách hiệu quả. Mặc dù vậy, trong quá trình thực hiện đề tài khó tránh khỏi sai sót, mong thầy cô đánh giá và nhận xét để em có thể hoàn thành những đề tài sau được tốt hơn. Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 20 tháng 3 năm 2026

Sinh viên

Phạm Vũ Hoàng Phúc

TÓM TẮT ĐỀ TÀI

Đề tài tập trung vào việc thiết kế, mô phỏng và hiện thực một bộ xử lý CPU 32-bit theo kiến trúc tập lệnh RISC-V RV32I trên kit FPGA PYNQ-Z2. Thiết kế được mô tả ở mức RTL bằng ngôn ngữ SystemVerilog và được kiểm tra, tổng hợp, thực thi trên phần mềm Vivado.

CPU được xây dựng từ các khối chức năng cơ bản như Program Counter, Instruction Memory, Register File, Immediate Generator, ALU, Branch Comparator, Load Store Unit và Control Unit. Từ nền tảng CPU đơn chu kỳ, thiết kế được mở rộng sang kiến trúc pipeline 5 giai đoạn gồm Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM) và Write Back (WB). Trong kiến trúc pipeline, mỗi lệnh cần đi qua nhiều chu kỳ clock để hoàn thành, nhưng nhiều lệnh khác nhau có thể được xử lý chồng lấp tại các giai đoạn khác nhau, giúp cải thiện thông lượng thực thi lệnh so với kiến trúc đơn chu kỳ.

Kết quả thực nghiệm cho thấy CPU có thể thực thi chương trình kiểm thử, đọc trạng thái button và điều khiển LED trên kit PYNQ-Z2. Kết quả synthesis/implementation cũng cho thấy thiết kế thỏa mãn timing với các ràng buộc đã đặt ra, chứng minh thiết kế có thể hoạt động trên phần cứng FPGA.

MỤC LỤC

1	GIỚI THIỆU	1
1.1	Tổng quan đề tài	1
1.2	Mục tiêu đề tài	1
1.3	Phạm vi thực hiện	2
1.4	Công cụ và thiết bị sử dụng	2
1.5	Cấu trúc báo cáo	2
2	CƠ SỞ LÝ THUYẾT	3
2.1	Kiến trúc tập lệnh ISA	3
2.2	Tổng quan về kiến trúc RISC-V	3
2.3	Hệ thống thanh ghi trong RISC-V	4
2.4	Định dạng lệnh trong RISC-V và tập lệnh RV32I	4
2.4.1	Định dạng lệnh trong RISC-V	4
2.4.2	Tập lệnh RV32I	5
2.5	Lý thuyết về xung đột (Pipeline Hazard)	5
2.5.1	Xung đột cấu trúc (Structural hazard)	5
2.5.2	Xung đột dữ liệu (Data hazard)	6
2.5.3	Xung đột điều khiển (Control Hazard)	7
3	THIẾT KẾ CPU RISC-V RV32I	8
3.1	Mô hình kiến trúc CPU đơn chu kì	8
3.1.1	Khối Arithmetic Logic Unit (ALU)	8
3.1.2	Khối Branch Compare (BRC)	10
3.1.3	Khối Register File (REGFILE)	12
3.1.4	Khối Memory	13
3.1.5	Khối Load Store Unit (LSU)	15
3.1.6	Khối Control Unit (CU)	16
3.1.7	Single cycle datapath	19
3.2	Chuyển đổi từ kiến trúc đơn chu kỳ sang kiến trúc pipeline	20
3.2.1	Thanh ghi IF/ID	21
3.2.2	Thanh ghi ID/EX	21
3.2.3	Thanh ghi EX/MEM	21
3.2.4	Thanh ghi MEM/WB	22
3.2.5	Control Unit trong kiến trúc pipeline	22
3.2.6	Branch Unit	23
3.2.7	Hazard Unit	23
3.2.8	Forwarding Unit	24
4	HIỆN THỰC CPU PIPELINE TRÊN KIT PYNQ-Z2	27
4.1	Mục tiêu hiện thực trên FPGA	27
4.2	Tổng quan module top	27
4.3	Thiết kế bộ chia clock	28

4.4	Thiết kế reset	28
4.5	Đồng bộ tín hiệu button vào CPU	29
4.6	Kết nối CPU pipeline với ngoại vi	29
4.7	File ràng buộc chân XDC	30
4.7.1	Ràng buộc clock	30
4.7.2	Ràng buộc button	30
4.7.3	Ràng buộc LED	30
4.8	Chương trình kiểm thử LED	30
4.9	Kết quả hiện thực trên kit PYNQ-Z2	31
4.10	Kết quả synthesis	31
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	33
5.1	Kết quả đạt được	33
5.2	Hạn chế	33
5.3	Hướng phát triển	33
	TÀI LIỆU THAM KHẢO	35

DANH MỤC ẢNH MINH HỌA

2.4.1	Các định dạng lệnh cơ sở trong RV32I	4
3.1.1	CPU RISC-V đơn chu kì	8
3.1.2	Các phép toán ALU	9
3.1.3	Sơ đồ khối ALU	9
3.1.4	ALU schematic	10
3.1.5	ALU simulation	10
3.1.6	Sơ đồ khối BRC	11
3.1.7	BRC schematic	11
3.1.8	BRC simulation	12
3.1.9	Sơ đồ khối REGFILE	13
3.1.10	REGFILE schematic	13
3.1.11	REGFILE simulation	13
3.1.12	IMEM schematic	14
3.1.13	IMEM simulation	14
3.1.14	Sơ đồ khối LSU	15
3.1.15	LSU schematic	16
3.1.16	CU schematic	19
3.1.17	SCDP	20
3.2.1	CPU pipeline với khối phát hiện Hazard và khối Forwarding	21
4.10.1	Kết quả sử dụng tài nguyên FPGA của thiết kế	32
4.10.2	Kết quả sử dụng tài nguyên theo từng module trong thiết kế	32
4.10.3	Kết quả phân tích timing của thiết kế	32

DANH MỤC BẢNG

1.4.1	Công cụ và thiết bị sử dụng trong đề tài	2
2.2.1	Một số biến thể và phần mở rộng thường gặp của RISC-V	3
2.3.1	Một số thanh ghi thường dùng trong RISC-V	4
2.4.1	Các nhóm lệnh chính trong tập lệnh RV32I	5
3.1.1	Đặc tả cổng vào/ra của khối ALU	8
3.1.2	Đặc tả cổng vào/ra của khối BRC	11
3.1.3	Đặc tả cổng vào/ra của khối Register File	12
3.1.4	Đặc tả cổng vào/ra của khối Memory	14
3.1.5	Đặc tả cổng vào/ra của khối LSU	15
3.1.6	Đặc tả cổng vào/ra của khối Control Unit	16
3.2.1	Đặc tả tín hiệu của khối Control Unit	22
3.2.2	Branch Unit Specification	23
3.2.3	Hazard Unit Specification	24
3.2.4	Forwarding Unit Specification	25
3.2.5	Mã điều khiển của tín hiệu forwarding	25
4.2.1	Đặc tả cổng vào/ra của module <code>top_pynq_pipeline</code>	27
4.6.1	Ánh xạ tín hiệu ngoại vi trong module <code>top</code>	29
4.7.1	Ràng buộc chân cho button trên PYNQ-Z2	30
4.7.2	Ràng buộc chân cho LED trên PYNQ-Z2	30
4.8.1	Các chế độ hoạt động của chương trình kiểm thử LED	31
4.9.1	Kết quả kiểm chứng chương trình LED trên PYNQ-Z2	31

DANH MỤC CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
CPU	Central Processing Unit – Bộ xử lý trung tâm
ISA	Instruction Set Architecture – Kiến trúc tập lệnh
RISC-V	Kiến trúc tập lệnh mở thuộc nhóm RISC
RV32I	RISC-V 32-bit Integer Base ISA
FPGA	Field Programmable Gate Array – Mạch logic khả trình
RTL	Register Transfer Level – Mức chuyển thanh ghi
ALU	Arithmetic Logic Unit – Khối số học và logic
BRC	Branch Compare – Khối so sánh rẽ nhánh
CU	Control Unit – Khối điều khiển
PC	Program Counter – Bộ đếm chương trình
IMEM	Instruction Memory – Bộ nhớ lệnh
DMEM	Data Memory – Bộ nhớ dữ liệu
LSU	Load Store Unit – Khối xử lý load/store
IF	Instruction Fetch – Giai đoạn lấy lệnh
ID	Instruction Decode – Giai đoạn giải mã lệnh
EX	Execute – Giai đoạn thực thi
MEM	Memory Access – Giai đoạn truy cập bộ nhớ
WB	Write Back – Giai đoạn ghi kết quả
MMIO	Memory-Mapped I/O – Ánh xạ ngoại vi vào bộ nhớ
XDC	Xilinx Design Constraints – File ràng buộc Vivado
LUT	Look-Up Table – Tài nguyên logic FPGA
FF	Flip-Flop – Phần tử nhớ trong FPGA
BRAM	Block RAM – Bộ nhớ khối trong FPGA
DSP	Digital Signal Processing – Khối xử lý số học chuyên dụng

Chương 1

GIỚI THIỆU

1.1 Tổng quan đề tài

Trong các hệ thống máy tính và thiết bị nhúng hiện nay, bộ xử lý trung tâm đóng vai trò quan trọng trong việc thực thi chương trình, xử lý dữ liệu và điều khiển hoạt động của toàn bộ hệ thống. Việc nghiên cứu và thiết kế CPU giúp người học hiểu rõ hơn nguyên lý hoạt động của máy tính ở mức phần cứng, từ quá trình nạp lệnh, giải mã lệnh, thực thi lệnh đến ghi kết quả.

RISC-V là một kiến trúc tập lệnh mở, được xây dựng dựa trên nguyên lý RISC với tập lệnh đơn giản, dễ mở rộng và phù hợp cho mục đích học tập, nghiên cứu cũng như phát triển phần cứng. Trong đó, RV32I là tập lệnh số nguyên cơ sở 32-bit, bao gồm các nhóm lệnh cơ bản như lệnh số học, logic, truy cập bộ nhớ và rẽ nhánh. Do đó, RV32I phù hợp để triển khai một CPU ở mức cơ bản trong phạm vi đồ án môn học.

Bên cạnh mô phỏng bằng phần mềm, việc kiểm chứng thiết kế trên FPGA giúp đánh giá hoạt động của CPU trên phần cứng thực tế. Kit PYNQ-Z2 có sẵn các tài nguyên như clock, nút nhấn, công tắc và LED, phù hợp để xây dựng các chương trình kiểm thử trực quan. Vì vậy, đề tài **“Thiết kế CPU theo cấu trúc RISC-V đa chu kỳ theo kiến trúc pipeline trên kit FPGA PYNQ-Z2”** được lựa chọn nhằm kết hợp giữa kiến thức kiến trúc máy tính, thiết kế phần cứng số và kiểm chứng thực nghiệm trên FPGA.

1.2 Mục tiêu đề tài

Mục tiêu của đề tài là thiết kế và kiểm chứng một CPU 32-bit theo kiến trúc tập lệnh RISC-V RV32I bằng ngôn ngữ mô tả phần cứng SystemVerilog trên phần mềm Vivado, sau đó hiện thực trên kit PYNQ-Z2.

Các mục tiêu cụ thể gồm:

- Tìm hiểu kiến trúc tập lệnh RISC-V RV32I, bao gồm hệ thống thanh ghi, định dạng lệnh và các nhóm lệnh cơ bản.
- Thiết kế các khối chức năng chính của CPU như Program Counter, Instruction Memory, Register File, Immediate Generator, ALU, Data Memory và Write Back.
- Tổ chức CPU theo kiến trúc pipeline gồm các giai đoạn IF, ID, EX, MEM và WB.
- Tích hợp các khối Hazard Unit và Forwarding Unit để xử lý một số xung đột dữ liệu trong pipeline.
- Mô phỏng chức năng CPU trên Vivado để kiểm tra hoạt động của các lệnh.
- Hiện thực thiết kế trên kit PYNQ-Z2 và kiểm chứng thông qua chương trình điều khiển LED bằng button.

1.3 Phạm vi thực hiện

Trong phạm vi đồ án, CPU được thiết kế ở mức RTL bằng SystemVerilog và được mô phỏng, tổng hợp, hiện thực trên phần mềm Vivado. Thiết kế tập trung vào tập lệnh RV32I cơ bản, bao gồm một số nhóm lệnh chính như lệnh số học, logic, dịch bit, load/store và branch/jump.

CPU được tổ chức theo kiến trúc pipeline 5 giai đoạn: Instruction Fetch, Instruction Decode, Execute, Memory Access và Write Back. Đề tài có xét đến xử lý xung đột dữ liệu bằng forwarding và stall ở mức cơ bản. Việc kiểm chứng trên kit PYNQ-Z2 được thực hiện thông qua các ngoại vi đơn giản. Trong đó, nút nhấn được dùng làm tín hiệu đầu vào hoặc reset, còn LED được dùng để hiển thị kết quả xử lý của chương trình chạy trên CPU.

1.4 Công cụ và thiết bị sử dụng

Các công cụ và thiết bị sử dụng trong đề tài được trình bày trong Bảng 1.4.1.

Bảng 1.4.1: Công cụ và thiết bị sử dụng trong đề tài

Thành phần	Nội dung
Ngôn ngữ thiết kế	SystemVerilog/ Assembly
Phần mềm thiết kế	Xilinx Vivado
Kit kiểm chứng	PYNQ-Z2
FPGA	Zynq-7000 XC7Z020
File ràng buộc	.xdc
File chương trình	.mem
Ngõ vào kiểm chứng	Clock, reset, button
Ngõ ra kiểm chứng	LED

Vivado được sử dụng để viết mã nguồn, mô phỏng RTL, tổng hợp, implementation, phân tích timing, generate bitstream và nạp thiết kế xuống kit FPGA. File .xdc được dùng để ánh xạ các tín hiệu trong thiết kế với chân vật lý trên kit PYNQ-Z2. File .mem được dùng để khởi tạo chương trình trong bộ nhớ lệnh của CPU.

1.5 Cấu trúc báo cáo

Báo cáo gồm các chương chính sau:

Chương 1: Tổng quan đề tài trình bày lý do chọn đề tài, mục tiêu, phạm vi thực hiện, công cụ sử dụng và cấu trúc báo cáo.

Chương 2: Cơ sở lý thuyết trình bày tổng quan về kiến trúc RISC-V, tập lệnh RV32I, định dạng lệnh, tập thanh ghi, kiến trúc pipeline và các loại hazard trong CPU pipeline.

Chương 3: Thiết kế CPU RISC-V RV32I trình bày kiến trúc tổng thể của CPU và mô tả chi tiết các khối chức năng như IF, ID, EX, MEM, WB, Hazard Unit và Forwarding Unit.

Chương 4: Hiện thực trên kit PYNQ-Z2 trình bày quá trình thiết kế module top, chia clock, ánh xạ button/LED, xây dựng file .xdc, nạp chương trình và generate bitstream.

Chương 5: Kết luận và hướng phát triển tổng kết kết quả đạt được, nêu hạn chế của thiết kế và đề xuất các hướng phát triển tiếp theo.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Kiến trúc tập lệnh ISA

Kiến trúc tập lệnh, hay Instruction Set Architecture, là lớp giao tiếp giữa phần mềm và phần cứng của bộ xử lý. ISA quy định các lệnh mà CPU có thể thực thi, cách mã hóa lệnh, hệ thống thanh ghi, kiểu dữ liệu, cơ chế truy cập bộ nhớ và cách điều khiển luồng chương trình.

Có hai hướng kiến trúc tập lệnh phổ biến là CISC và RISC. CISC sử dụng tập lệnh phức tạp, mỗi lệnh có thể thực hiện nhiều thao tác, tuy nhiên phần cứng giải mã thường phức tạp hơn. Ngược lại, RISC sử dụng tập lệnh đơn giản, định dạng lệnh rõ ràng, thuận lợi cho việc thiết kế pipeline và tối ưu phần cứng. RISC-V là một kiến trúc tập lệnh thuộc nhóm RISC.

2.2 Tổng quan về kiến trúc RISC-V

RISC-V là một kiến trúc tập lệnh mở, được thiết kế dựa trên nguyên lý RISC (*Reduced Instruction Set Computer*). Điểm nổi bật của RISC-V là đặc tả mở, cho phép sử dụng, nghiên cứu, triển khai và mở rộng mà không phụ thuộc vào giấy phép thương mại như một số kiến trúc tập lệnh khác. Do đó, RISC-V được sử dụng rộng rãi trong giáo dục, nghiên cứu kiến trúc máy tính và phát triển các bộ xử lý tùy biến.

RISC-V được tổ chức theo hướng gồm một tập lệnh cơ sở và các phần mở rộng tùy chọn. Tập lệnh cơ sở quy định các lệnh tối thiểu để xây dựng một CPU có khả năng hoạt động, trong khi các phần mở rộng bổ sung thêm các chức năng như nhân chia số nguyên, xử lý số thực, lệnh nén hoặc xử lý nguyên tử. Các đặc tả chính thức của RISC-V được RISC-V International công bố công khai và duy trì.

Một số biến thể và phần mở rộng thường gặp của RISC-V được trình bày trong Bảng 2.2.1.

Bảng 2.2.1: Một số biến thể và phần mở rộng thường gặp của RISC-V

Ký hiệu	Ý nghĩa
RV32I	Tập lệnh số nguyên cơ sở 32-bit
RV64I	Tập lệnh số nguyên cơ sở 64-bit
M	Mở rộng lệnh nhân và chia số nguyên
A	Mở rộng lệnh nguyên tử
F	Mở rộng số thực đơn chính xác
D	Mở rộng số thực kép chính xác
C	Mở rộng lệnh nén 16-bit

2.3 Hệ thống thanh ghi trong RISC-V

RISC-V RV32I có 32 thanh ghi đa dụng, được đánh số từ `x0` đến `x31`. Mỗi thanh ghi có độ rộng 32 bit. Thanh ghi `x0` được nối cứng bằng 0, vì vậy mọi thao tác ghi vào `x0` đều không làm thay đổi giá trị của thanh ghi này. Đây là đặc điểm giúp đơn giản hóa nhiều lệnh, chẳng hạn như gán giá trị 0, so sánh với 0 hoặc tạo lệnh giả.

Ngoài tên vật lý từ `x0` đến `x31`, các thanh ghi còn có tên ABI để thuận tiện cho lập trình hợp ngữ. Một số thanh ghi thường dùng được trình bày trong Bảng 2.3.1.

Bảng 2.3.1: Một số thanh ghi thường dùng trong RISC-V

Thanh ghi	Tên ABI	Chức năng
<code>x0</code>	<code>zero</code>	Luôn có giá trị 0
<code>x1</code>	<code>ra</code>	Lưu địa chỉ trả về
<code>x2</code>	<code>sp</code>	Con trỏ ngăn xếp
<code>x5-x7</code>	<code>t0-t2</code>	Thanh ghi tạm
<code>x8-x9</code>	<code>s0-s1</code>	Thanh ghi lưu trữ
<code>x10-x17</code>	<code>a0-a7</code>	Đối số hoặc giá trị trả về
<code>x18-x27</code>	<code>s2-s11</code>	Thanh ghi lưu trữ
<code>x28-x31</code>	<code>t3-t6</code>	Thanh ghi tạm

Trong thiết kế CPU, hệ thống thanh ghi được hiện thực bằng khối *Register File*. Khối này có hai cổng đọc và một cổng ghi. Hai cổng đọc dùng để lấy dữ liệu từ hai thanh ghi nguồn `rs1` và `rs2`, còn cổng ghi dùng để ghi kết quả vào thanh ghi đích `rd`.

2.4 Định dạng lệnh trong RISC-V và tập lệnh RV32I

2.4.1 Định dạng lệnh trong RISC-V

Các lệnh được chia thành nhiều định dạng khác nhau nhằm phù hợp với từng nhóm chức năng, bao gồm R-type, I-type, S-type, B-type, U-type và J-type. Việc chuẩn hóa độ dài lệnh giúp quá trình nạp lệnh, giải mã lệnh và thiết kế khối điều khiển trở nên đơn giản hơn. Đồng thời, các trường như `opcode`, `rd`, `rs1`, `rs2`, `funct3`, `funct7` và `imm` được bố trí tương đối nhất quán, tạo thuận lợi cho việc hiện thực phần cứng.

31–25	24–20	19–15	14–12	11–7	6–0	Dạng
<code>funct7</code>	<code>rs2</code>	<code>rs1</code>	<code>funct3</code>	<code>rd</code>	<code>opcode</code>	R-type
<code>imm[11:0]</code>		<code>rs1</code>	<code>funct3</code>	<code>rd</code>	<code>opcode</code>	I-type
<code>imm[11:5]</code>	<code>rs2</code>	<code>rs1</code>	<code>funct3</code>	<code>imm[4:0]</code>	<code>opcode</code>	S-type
<code>imm[12 10:5]</code>	<code>rs2</code>	<code>rs1</code>	<code>funct3</code>	<code>imm[4:1 11]</code>	<code>opcode</code>	B-type
<code>imm[31:12]</code>				<code>rd</code>	<code>opcode</code>	U-type
<code>imm[20 10:1 11 19:12]</code>				<code>rd</code>	<code>opcode</code>	J-type

Hình 2.4.1: Các định dạng lệnh cơ sở trong RV32I

Ý nghĩa của các định dạng lệnh được trình bày như sau:

- **R-type:** được sử dụng cho các lệnh tính toán số học và logic giữa hai thanh ghi nguồn. Các lệnh tiêu biểu gồm `add`, `sub`, `and`, `or`, `xor`, `sll`, `srl`.

- **I-type:** được sử dụng cho các lệnh tính toán với hằng số tức thời, các lệnh tải dữ liệu từ bộ nhớ và lệnh nhảy gián tiếp. Các lệnh tiêu biểu gồm `addi`, `andi`, `ori`, `lw` và `jalr`.
- **S-type:** được sử dụng cho các lệnh lưu dữ liệu từ thanh ghi xuống bộ nhớ. Các lệnh tiêu biểu gồm `sw`, `sh` và `sb`.
- **B-type:** được sử dụng cho các lệnh rẽ nhánh có điều kiện. CPU sẽ so sánh hai thanh ghi nguồn và cập nhật giá trị PC nếu điều kiện rẽ nhánh thỏa mãn. Các lệnh tiêu biểu gồm `beq`, `bne`, `blt` và `bge`.
- **U-type:** được sử dụng cho các lệnh tạo hằng số lớn hoặc tính địa chỉ tương đối với PC. Các lệnh thuộc định dạng này gồm `lui` và `auipc`.
- **J-type:** được sử dụng cho lệnh nhảy không điều kiện `jal`. Lệnh này vừa cập nhật PC đến địa chỉ đích, vừa lưu địa chỉ lệnh kế tiếp vào thanh ghi đích.

2.4.2 Tập lệnh RV32I

RV32I là tập lệnh số nguyên cơ sở của RISC-V dành cho bộ xử lý 32-bit. Trong RV32I, độ rộng dữ liệu của các thanh ghi là 32 bit, còn mỗi lệnh cơ bản có độ dài 32 bit. Các nhóm lệnh chính trong RV32I được trình bày trong Bảng 2.4.1.

Bảng 2.4.1: Các nhóm lệnh chính trong tập lệnh RV32I

Nhóm lệnh	Chức năng	Ví dụ
Lệnh số học và logic	Thực hiện các phép toán số học, logic, dịch bit và so sánh giữa các thanh ghi	<code>add</code> , <code>sub</code> , <code>and</code> , <code>or</code> , <code>xor</code> , <code>sll</code> , <code>srl</code> , <code>sra</code> , <code>slt</code> , <code>sltu</code>
Lệnh tức thời	Thực hiện phép toán với một toán hạng là giá trị tức thời được mã hóa trực tiếp trong lệnh	<code>addi</code> , <code>andi</code> , <code>ori</code> , <code>xori</code>
Lệnh truy cập bộ nhớ	Đọc dữ liệu từ bộ nhớ vào thanh ghi hoặc ghi dữ liệu từ thanh ghi xuống bộ nhớ	<code>lw</code> , <code>sw</code> , <code>lb</code> , <code>lh</code> , <code>sb</code> , <code>sh</code>
Lệnh rẽ nhánh	Thay đổi luồng thực thi chương trình dựa trên kết quả so sánh giữa hai thanh ghi	<code>beq</code> , <code>bne</code> , <code>blt</code> , <code>bge</code> , <code>bltu</code> , <code>bgeu</code>
Lệnh tức thời 20-bit cao	Nạp giá trị tức thời 20-bit vào phần cao của thanh ghi hoặc tính địa chỉ tương đối với PC	<code>lui</code> , <code>auipc</code>
Lệnh nhảy	Thực hiện nhảy không điều kiện hoặc gọi chương trình con, đồng thời lưu địa chỉ quay lại	<code>jal</code> , <code>jalr</code>

2.5 Lý thuyết về xung đột (Pipeline Hazard)

Xung đột là trạng thái mà lệnh tiếp theo không thể thực thi trong chu kỳ pipeline ngay sau đó, hoặc vẫn thực thi nhưng có thể cho ra kết quả sai.

2.5.1 Xung đột cấu trúc (Structural hazard)

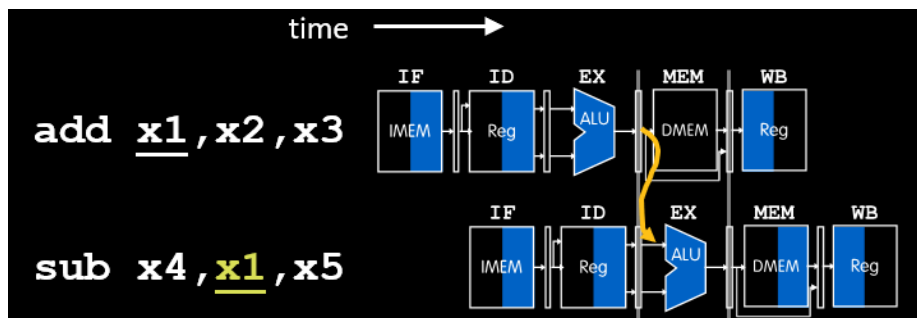
Xung đột cấu trúc xảy ra khi nhiều giai đoạn pipeline cần sử dụng cùng một tài nguyên phần cứng trong cùng một chu kỳ, nhưng tài nguyên đó không hỗ trợ nhiều truy cập đồng thời.

Nếu bộ nhớ lệnh và bộ nhớ dữ liệu dùng chung một phần cứng bộ nhớ, tầng IF có thể cần đọc lệnh trong cùng chu kỳ mà tầng MEM cần đọc hoặc ghi dữ liệu. Khi đó, pipeline phải tạm dừng một trong hai thao tác để tránh tranh chấp tài nguyên. Một cách xử lý phổ biến là tách riêng Instruction Memory và Data Memory, giúp tầng IF và tầng MEM có thể truy cập bộ nhớ độc lập.

2.5.2 Xung đột dữ liệu (Data hazard)

Xung đột dữ liệu xảy ra khi một lệnh cần sử dụng kết quả của lệnh trước đó, nhưng kết quả này chưa sẵn sàng tại thời điểm lệnh sau cần dùng. Đây là loại xung đột thường gặp trong CPU pipeline.

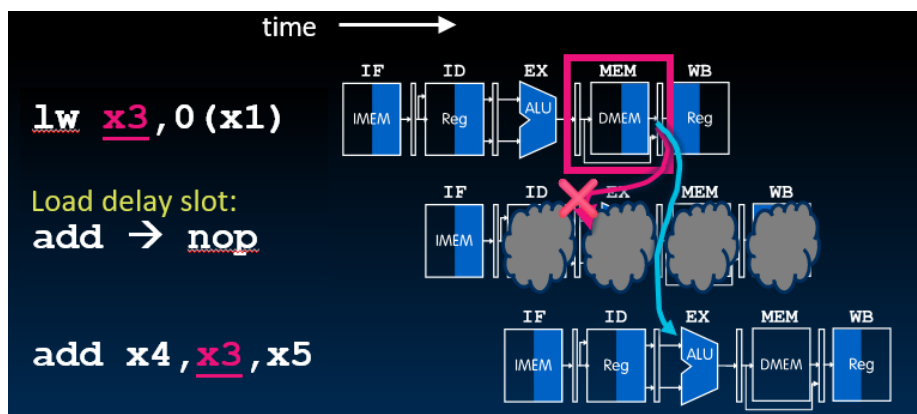
Ví dụ:



Trong đoạn lệnh trên, lệnh `add` tạo ra kết quả và ghi vào thanh ghi `x3`. Lệnh `sub` ngay sau đó lại cần sử dụng giá trị của `x3`. Tuy nhiên, trong pipeline, tại thời điểm lệnh `sub` đi đến tầng EX, kết quả của lệnh `add` có thể chưa được ghi về Register File ở tầng WB. Nếu không xử lý, lệnh `sub` có thể đọc giá trị cũ của `x3` và cho ra kết quả sai.

Đối với trường hợp kết quả đã được tạo ra ở ALU nhưng chưa ghi về Register File, có thể sử dụng kỹ thuật *forwarding*. Forwarding cho phép chuyển trực tiếp kết quả từ các tầng sau của pipeline, chẳng hạn EX/MEM hoặc MEM/WB, về đầu vào ALU của lệnh đang thực thi. Nhờ đó, lệnh phụ thuộc dữ liệu có thể nhận được toán hạng đúng mà không cần chờ đến khi dữ liệu được ghi về Register File.

Tuy nhiên, một số trường hợp không thể xử lý hoàn toàn bằng forwarding. Trường hợp điển hình là *load-use hazard*:



Ở lệnh `lw`, dữ liệu chỉ có sau khi truy cập bộ nhớ ở tầng MEM. Trong khi đó, lệnh `add` ngay sau đó cần sử dụng giá trị `x3` ở tầng EX. Do dữ liệu chưa sẵn sàng kịp thời, pipeline cần *stall* một chu kỳ để chờ dữ liệu hợp lệ. Khi *stall*, một số thanh ghi pipeline được giữ nguyên và một *bubble* được chèn vào pipeline để tránh lệnh sau sử dụng dữ liệu sai.

2.5.3 Xung đột điều khiển (Control Hazard)

Xung đột điều khiển xảy ra khi CPU gặp các lệnh làm thay đổi luồng thực thi chương trình, chẳng hạn như lệnh rẽ nhánh hoặc lệnh nhảy. Với các lệnh này, địa chỉ lệnh tiếp theo có thể là $PC + 4$ hoặc địa chỉ đích của lệnh branch/jump.

Ví dụ:

```
beq x1, x2, label  
addi x3, x0, 1
```

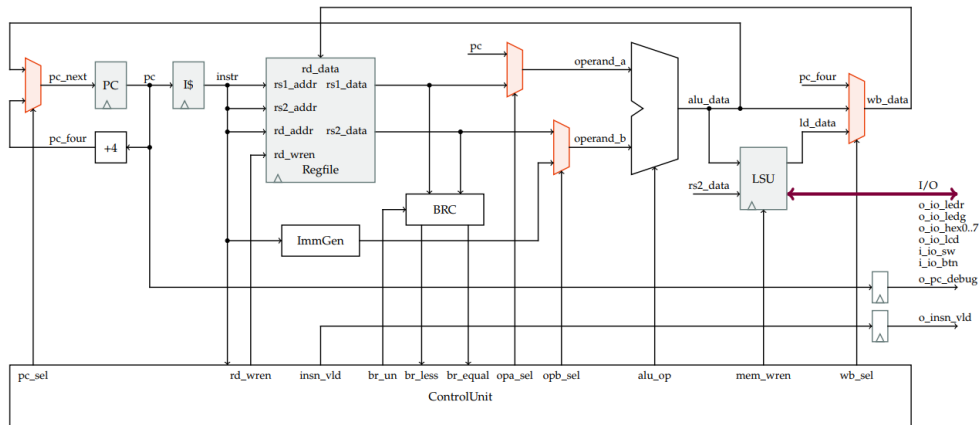
Nếu điều kiện $x1 == x2$ đúng, CPU sẽ cập nhật PC đến địa chỉ `label`. Tuy nhiên, trước khi điều kiện rẽ nhánh được xác định, pipeline có thể đã nạp lệnh kế tiếp là `addi x3, x0, 1`. Nếu branch được thực hiện, lệnh này không thuộc luồng thực thi đúng và cần bị loại bỏ.

Trong phạm vi thiết kế của đề tài, cơ chế được dùng để xử lý xung đột điều khiển là **flush**. Flush làm vô hiệu hóa các lệnh đã được nạp sai vào pipeline, đồng thời cập nhật PC đến địa chỉ đúng. Khi flush một lệnh, các tín hiệu điều khiển liên quan đến ghi thanh ghi hoặc ghi bộ nhớ phải được vô hiệu hóa để lệnh sai không làm thay đổi trạng thái của CPU. Trong thiết kế này, CPU chưa sử dụng branch prediction.

Chương 3

THIẾT KẾ CPU RISC-V RV32I

3.1 Mô hình kiến trúc CPU đơn chu kì



Hình 3.1.1: CPU RISC-V đơn chu kì

3.1.1 Khối Arithmetic Logic Unit (ALU)

- Module name: `alu.sv`
- I/O ports:

Bảng 3.1.1: Đặc tả cổng vào/ra của khối ALU

Signal name	Width	Direction	Description
<code>i_op_a</code>	32	input	First operand for ALU operations.
<code>i_op_b</code>	32	input	Second operand for ALU operations.
<code>i_alu_op</code>	4	input	The operation to be performed.
<code>o_alu_data</code>	32	output	Result of the ALU operation.

- Phân tích yêu cầu thiết kế:

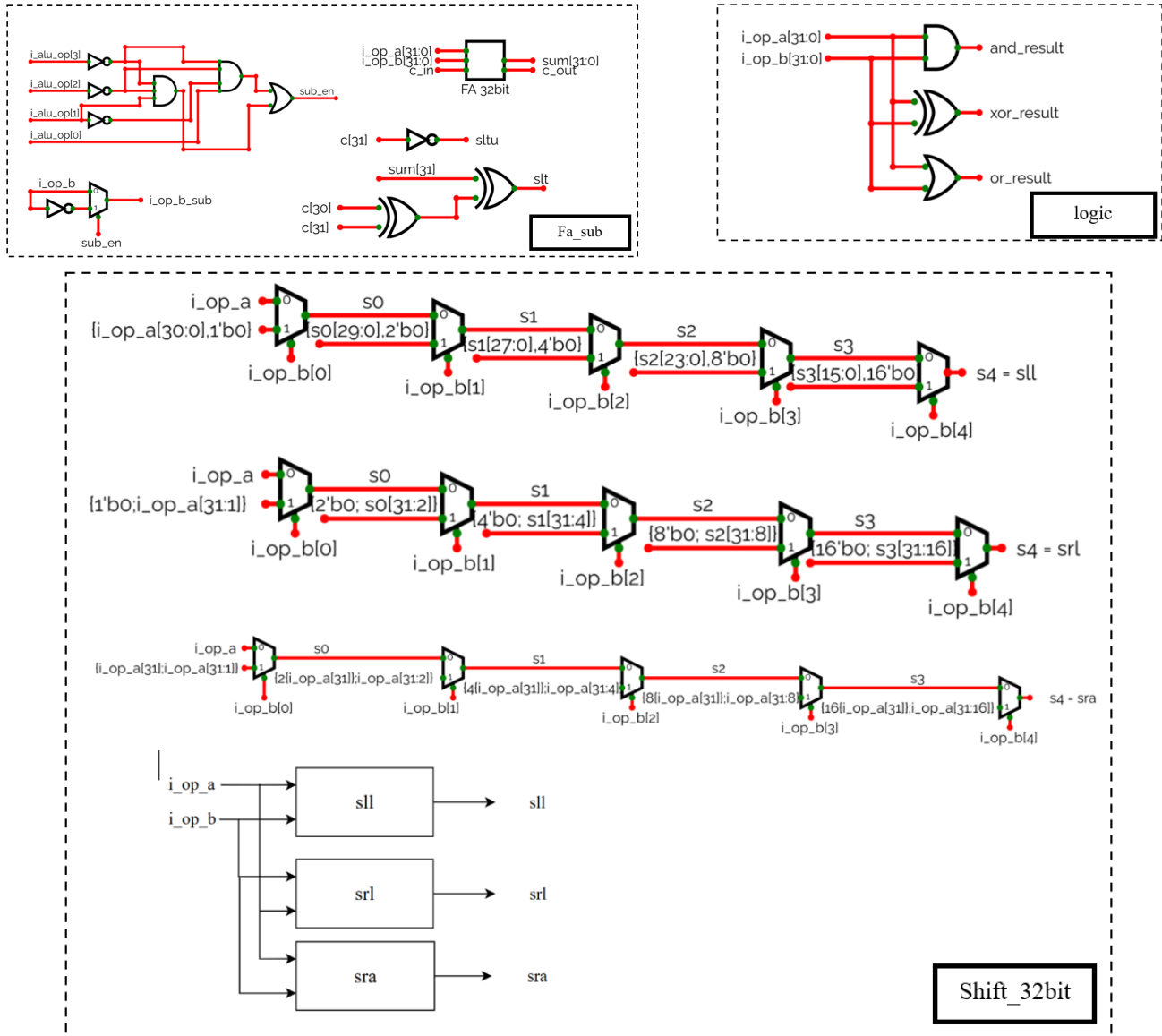
- Bộ ALU (Arithmetic Logic Unit) thực hiện các phép tính số học và logic trong tập lệnh RV32I. Bộ ALU nhận dữ liệu đầu vào từ thanh ghi, thực hiện các phép tính toán theo tín hiệu điều khiển và trả ra kết quả.

– Các phép toán cần thực hiện:

alu_op	Description (R-type)	Description (I-type)
ADD	$rd \leftarrow rs1 + rs2$	$rd \leftarrow rs1 + imm$
SUB	$rd \leftarrow rs1 - rs2$	n/a
SLT	$rd \leftarrow (rs1 < rs2)? 1 : 0$	$rd \leftarrow (rs1 < imm)? 1 : 0$
SLTU	$rd \leftarrow (rs1 < rs2)? 1 : 0$	$rd \leftarrow (rs1 < imm)? 1 : 0$
XOR	$rd \leftarrow rs1 \oplus rs2$	$rd \leftarrow rs1 \oplus imm$
OR	$rd \leftarrow rs1 \vee rs2$	$rd \leftarrow rs1 \vee imm$
AND	$rd \leftarrow rs1 \wedge rs2$	$rd \leftarrow rs1 \wedge imm$
SLL	$rd \leftarrow rs1 \ll rs2[4:0]$	$rd \leftarrow rs1 \ll imm[4:0]$
SRL	$rd \leftarrow rs1 \gg rs2[4:0]$	$rd \leftarrow rs1 \gg imm[4:0]$
SRA	$rd \leftarrow rs1 \ggg rs2[4:0]$	$rd \leftarrow rs1 \ggg imm[4:0]$

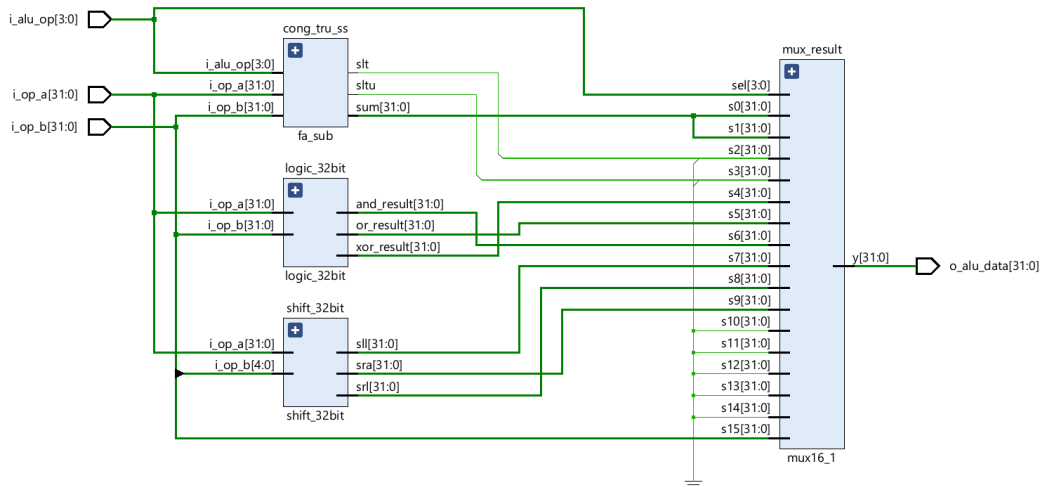
Hình 3.1.2: Các phép toán ALU

• Sơ đồ khối:



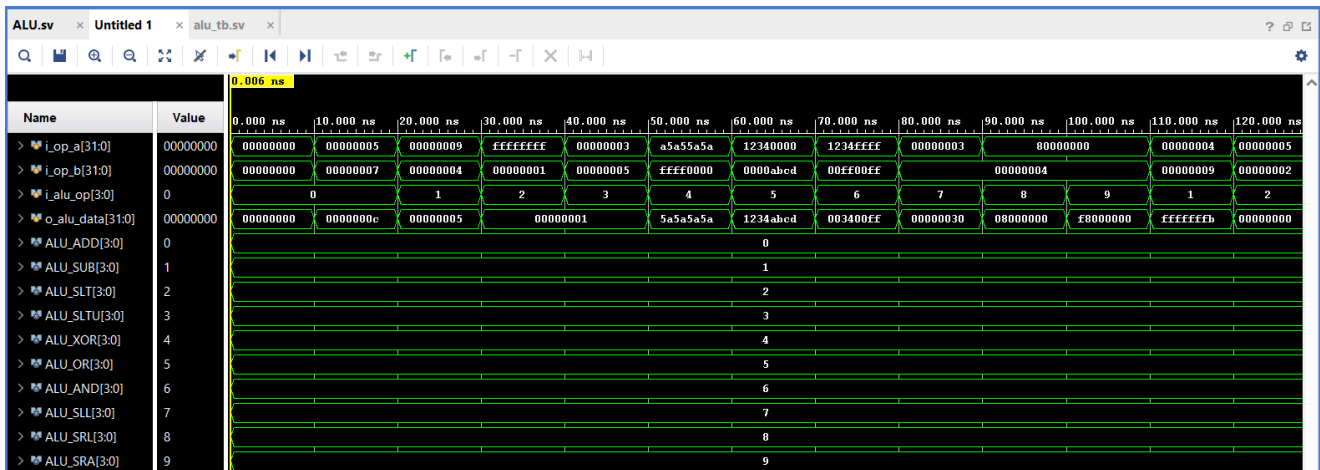
Hình 3.1.3: Sơ đồ khối ALU

- Schematic:



Hình 3.1.4: ALU schematic

- Simulation:



Hình 3.1.5: ALU simulation

3.1.2 Khối Branch Compare (BRC)

- Module name: brc sv

- I/O ports:

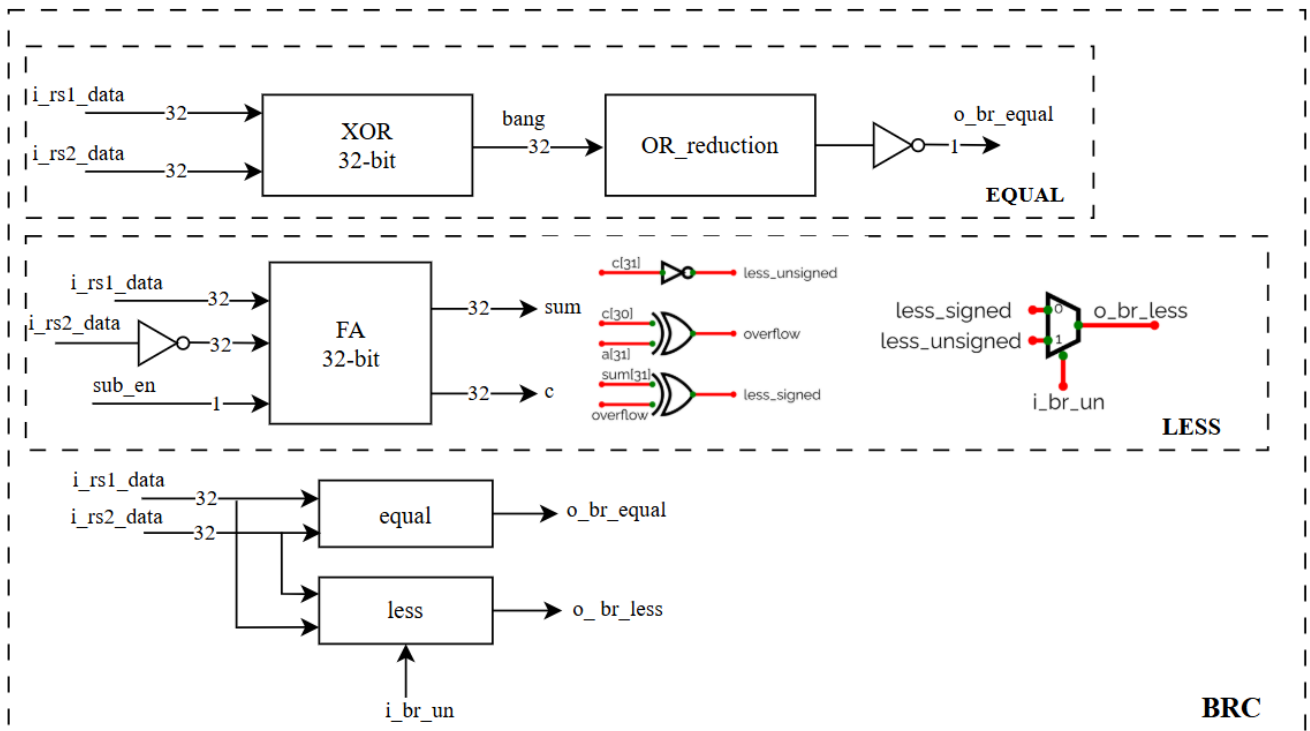
- Phân tích yêu cầu thiết kế BRC

- Bộ BRC (Branch Comparison) được thiết kế để thực hiện so sánh giá trị của hai thanh ghi `i_rs1_data` và `i_rs2_data` để xác định kết quả của các lệnh rẽ nhánh. Có hai đầu ra `br_less` nếu $rs1 < rs2$ (nếu `o_br_less` = 1 thì ra kết quả như trên, nếu `o_br_less` = 0 thì ngược lại $rs1 \geq rs2$) và `br_equal` nếu $rs1 = rs2$.
- Nếu tín hiệu `i_br_un` = 0, thực hiện so sánh không dấu (unsigned). Ngược lại, nếu `i_br_un` = 1, thực hiện so sánh có dấu (signed).

Bảng 3.1.2: Đặc tả cổng vào/ra của khối BRC

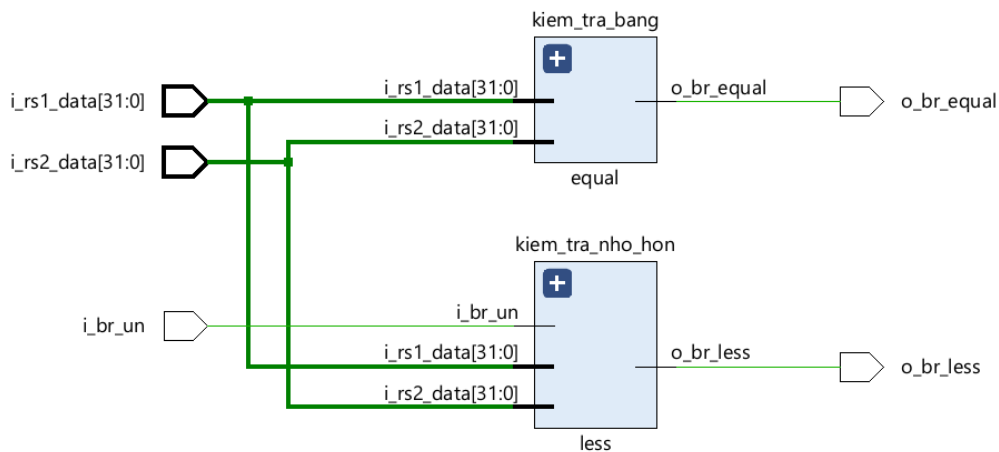
Signal name	Width	Direction	Description
i_rs1_data	32	input	Data from the first register.
i_rs2_data	32	input	Data from the second register.
i_br_un	1	input	Comparison mode (1 if unsigned, 0 if signed).
o_br_less	1	output	Output is 1 if $rs1 < rs2$.
o_br_equal	1	output	Output is 1 if $rs1 = rs2$.

• Sơ đồ khối:



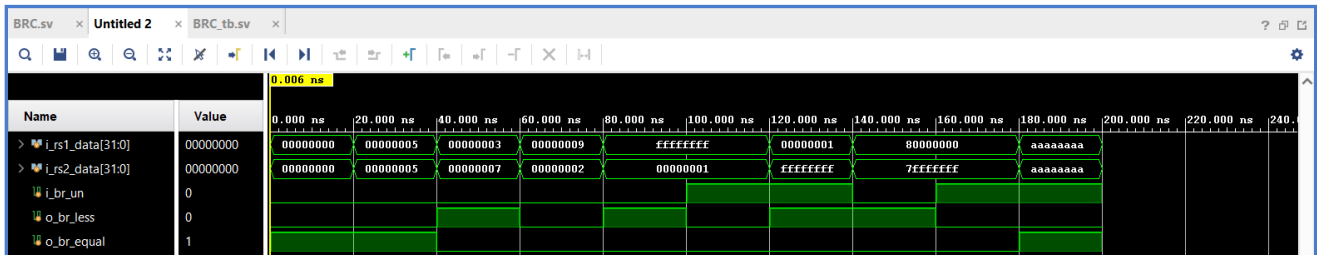
Hình 3.1.6: Sơ đồ khối BRC

• Schematic:



Hình 3.1.7: BRC schematic

- Simulation:



Hình 3.1.8: BRC simulation

3.1.3 Khối Register File (REGFILE)

- Module name: regfile.sv
- I/O ports:

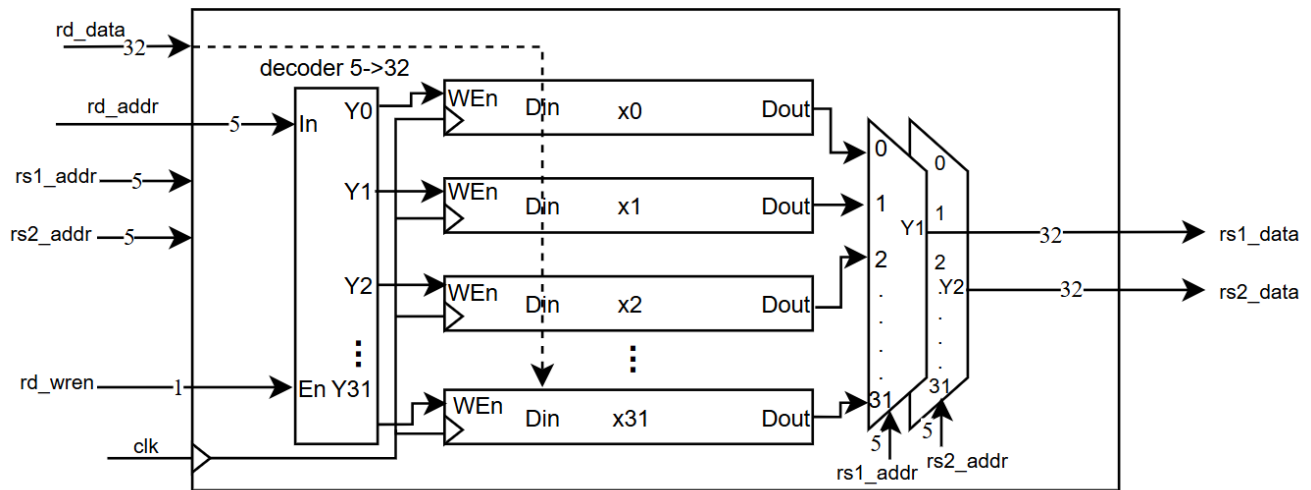
Bảng 3.1.3: Đặc tả cổng vào/ra của khối Register File

Signal name	Width	Direction	Description
i_clk	1	input	Global clock.
i_reset	1	input	Global active reset.
i_rs1_addr	5	input	Address of the first source register.
i_rs2_addr	5	input	Address of the second source register.
o_rs1_data	32	output	Data from the first source register.
o_rs2_data	32	output	Data from the second source register.
i_rd_addr	5	input	Address of the destination register.
i_rd_data	32	input	Data to write to the destination register.
i_rd_wren	1	input	Write enable for the destination register.

- Phân tích yêu cầu thiết kế Regfile

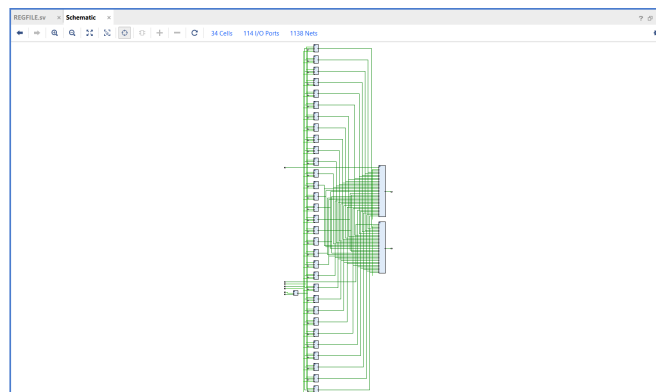
- Thiết kế một tập 32 thanh ghi, mỗi thanh ghi có độ rộng 32 bit, trong đó thanh ghi 0 luôn có giá trị 0. Tập thanh ghi này có hai cổng đọc và một cổng ghi. Regfile có chức năng xuất dữ liệu khi cung cấp địa chỉ từ **rs1** và **rs2**.
- Thiết kế tập thanh ghi gồm một bộ giải mã, 32 thanh ghi 32 bit và hai bộ chọn kênh đọc. Quá trình ghi được thực hiện đồng bộ theo xung **clk**; khi có cạnh tác động của xung clock và tín hiệu **i_rd_wren** được kích hoạt, dữ liệu đầu vào sẽ được ghi vào thanh ghi đích. Quá trình đọc được thực hiện bất đồng bộ, không phụ thuộc vào xung **clk**; dữ liệu tại các thanh ghi được chọn sẽ xuất hiện trực tiếp ở ngõ ra tương ứng.

- Sơ đồ khối:



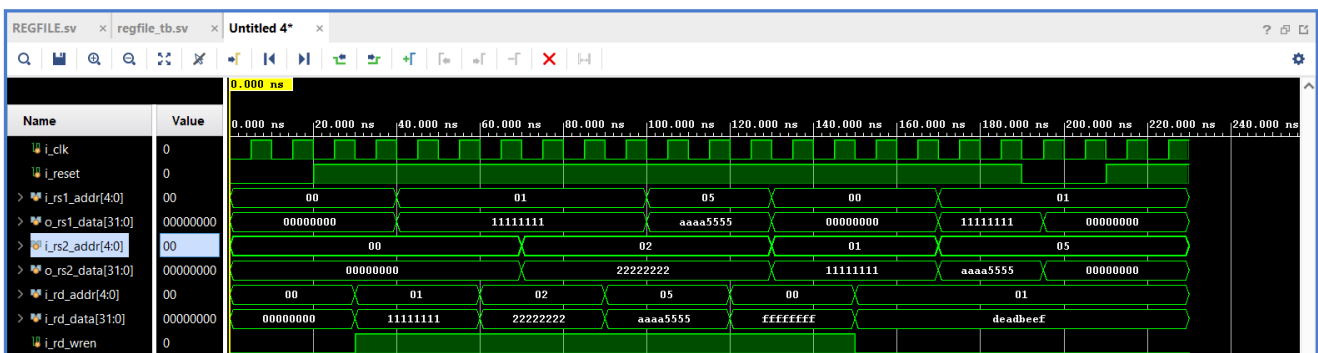
Hình 3.1.9: Sơ đồ khối REGFILE

- Schematic:



Hình 3.1.10: REGFILE schematic

- Simulation:



Hình 3.1.11: REGFILE simulation

3.1.4 Khối Memory

- Module name: imem.sv
- I/O ports:

Bảng 3.1.4: Đặc tả cổng vào/ra của khối Memory

Signal name	Width	Direction	Description
i_clk	1	input	Global clock.
i_addr	32	input	Address for both read and write operations.
o_rdata	32	output	Read data.

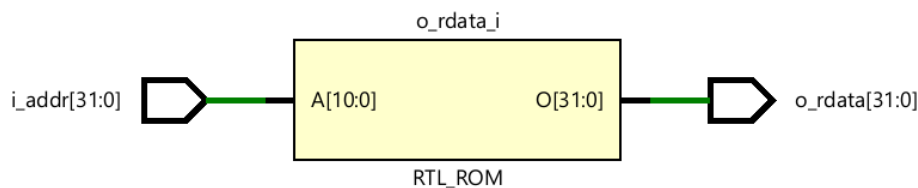
• Phân tích yêu cầu thiết kế Memory

- Thiết kế Memory gồm hai thành phần chính: Data Memory và Instruction Memory. Data Memory được tích hợp trong khối LSU để thực hiện chức năng đọc/ghi dữ liệu và giao tiếp với các ngoại vi. Instruction Memory có nhiệm vụ lưu trữ mã lệnh chương trình và xuất lệnh tương ứng với địa chỉ do PC cung cấp.
- Khối Instruction Memory được thiết kế với dung lượng 8 kB, tổ chức dưới dạng 2048 từ nhớ, mỗi từ có độ rộng 32 bit. Mỗi từ nhớ lưu trữ một lệnh RV32I 32-bit. Trong thiết kế, bộ nhớ lệnh được khai báo bằng mảng logic `[31:0] imem [0:2047]`, tương ứng với tổng dung lượng:

$$2048 \times 32 \text{ bit} = 65536 \text{ bit} = 8192 \text{ byte} = 8 \text{ kB}.$$

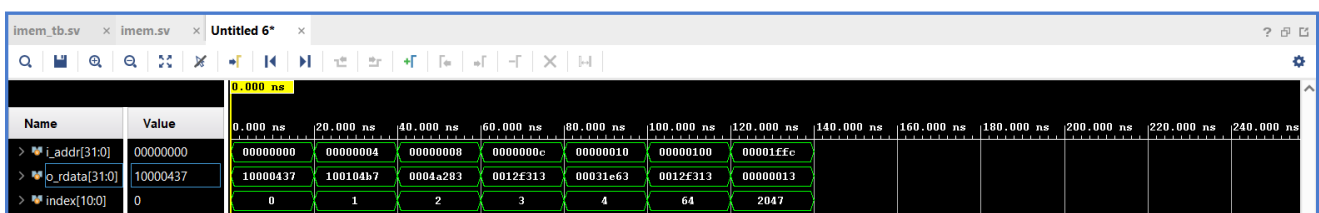
- Do PC xuất địa chỉ theo đơn vị byte, trong khi Instruction Memory được tổ chức theo từng word 32-bit, hai bit thấp của địa chỉ được loại bỏ khi truy xuất bộ nhớ lệnh. Vì vậy, địa chỉ đọc trong mảng được lấy bằng `i_addr[12:2]`. Cách ánh xạ này giúp PC có thể tăng 4 sau mỗi lệnh, nhưng vẫn truy xuất đúng từng lệnh 32-bit trong Instruction Memory. Ví dụ, `PC = 0` sẽ đọc `imem[0]`, `PC = 4` sẽ đọc `imem[1]`, `PC = 8` sẽ đọc `imem[2]`.
- Trước khi nạp chương trình, toàn bộ Instruction Memory được khởi tạo bằng mã lệnh `32'h00000013`, tương ứng với lệnh `nop = addi x0, x0, 0`. Việc này giúp tránh giá trị không xác định ở các ô nhớ chưa sử dụng. Sau đó, chương trình kiểm thử LED được nạp vào bộ nhớ lệnh bằng lệnh `$readmemh("lednhieuchedo.mem", imem)`.

• Schematic:



Hình 3.1.12: IMEM schematic

• Simulation:



Hình 3.1.13: IMEM simulation

3.1.5 Khối Load Store Unit (LSU)

- Module name: `lsu.sv`
- I/O ports:

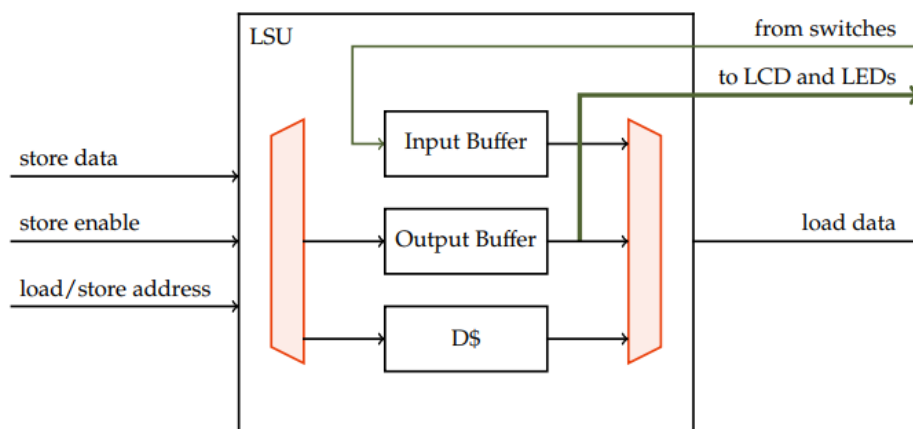
Bảng 3.1.5: Đặc tả cổng vào/ra của khối LSU

Signal name	Width	Direction	Description
<code>i_clk</code>	1	input	Global clock, active on the rising edge.
<code>i_reset</code>	1	input	Global active reset.
<code>i_lsu_addr</code>	32	input	Address for data read/write.
<code>i_st_data</code>	32	input	Data to be stored.
<code>i_lsu_wren</code>	1	input	Write enable signal (1 if writing).
<code>o_ld_data</code>	32	output	Data read from memory.
<code>o_io_ledr</code>	32	output	Output for red LEDs.
<code>o_io_ledg</code>	32	output	Output for green LEDs.
<code>o_io_hex0..7</code>	7	output	Output for 7-segment displays.
<code>o_io_lcd</code>	32	output	Output for the LCD register.
<code>i_io_sw</code>	32	input	Input for switches.
<code>type_sel</code>	2	input	Select store/load type.

- Phân tích yêu cầu thiết kế LSU

- Thiết kế khối LSU (Load-Store Unit) gồm:
 - + Bao gồm bộ nhớ Data Memory dung lượng 2 kB, Input Buffer gồm 1 thanh ghi và Output Buffer gồm 11 thanh ghi.
 - + Data Memory thực hiện chức năng đọc/ghi dữ liệu; Output Buffer thực hiện ghi dữ liệu đến các ngoại vi như LED, LCD và LED 7 đoạn; Input Buffer thực hiện đọc dữ liệu từ switches.
- Sử dụng mảng hai chiều để khai báo Data Memory gồm 512 word, mỗi word có độ rộng 32 bit.

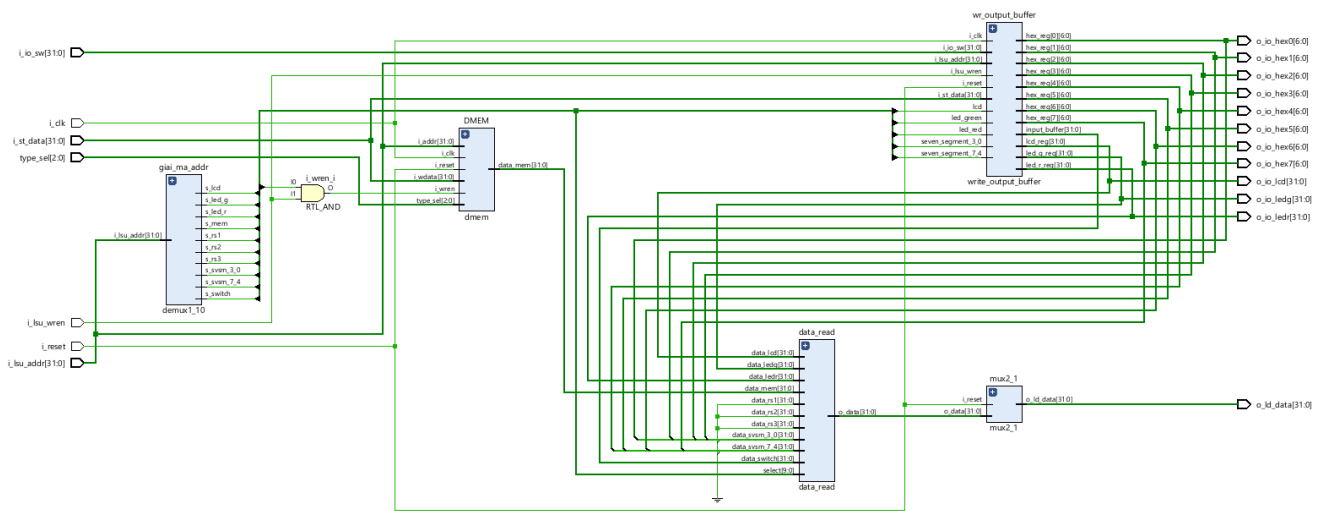
- Sơ đồ khối:



Hình 3.1.14: Sơ đồ khối LSU

CHƯƠNG 3. THIẾT KẾ CPU RISC-V RV32I

- **Schematic:**



Hình 3.1.15: LSU schematic

3.1.6 Khối Control Unit (CU)

- Module name: CU.sv
- I/O ports:

Bảng 3.1.6: Đặc tả cổng vào/ra của khối Control Unit

Signal name	Width	Direction	Description
instr	32	input	Instruction 32 bit.
br_less	1	input	Compare flag (1 if rs1 < rs2).
br_equal	1	input	Compare flag (1 if rs1 = rs2).
pc_sel	1	output	PC select (0 when PC+4; 1 when branch).
rd_wren	1	output	Write enable (1 if writing).
insn_vld	1	output	Instruction valid (1 if valid).
br_un	1	output	Compare unsigned select (1 if unsigned).
opa_sel	1	output	Operand select A (1 if rs1 , 0 if PC).
opb_sel	1	output	Operand select B (1 if immediate, 0 if rs2).
alu_op	4	output	Select ALU operation (add, sub, and, ...).
mem_wren	1	output	Write enable (1 if writing).
type_sel	3	output	Type select.
wb_sel	2	output	Write-back select.

- Phân tích yêu cầu thiết kế Control Unit

- tiến hành phân loại lệnh dựa trên trường **opcode** tại các bit [6:0]. Từ đó, tập lệnh được chia thành các nhóm chính gồm R-format, I-format, S-format, B-format, J-format và U-format.

- Trong từng nhóm lệnh, trường `funct3` được sử dụng để phân biệt các lệnh con. Đối với những trường hợp hai lệnh có cùng `opcode` và `funct3`, bit tương ứng trong trường `funct7` tiếp tục được sử dụng để xác định chính xác loại lệnh cần giải mã.

R-format:

funct7			funct3		opcode	
0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	001	rd	0110011	sll
0000000	rs2	rs1	010	rd	0110011	slt
0000000	rs2	rs1	011	rd	0110011	sltu
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	101	rd	0110011	srl
0100000	rs2	rs1	101	rd	0110011	sra
0000000	rs2	rs1	110	rd	0110011	or
0000000	rs2	rs1	111	rd	0110011	and

I-format layout:

			funct3	opcode		
imm[11:0]	rs1		000	rd	0010011	addi
imm[11:0]	rs1		010	rd	0010011	slti
imm[11:0]	rs1		011	rd	0010011	sltiu
imm[11:0]	rs1		100	rd	0010011	xori
imm[11:0]	rs1		110	rd	0010011	ori
imm[11:0]	rs1		111	rd	0010011	andi
0000000	shamt	rs1	001	rd	0010011	slli
0000000	shamt	rs1	101	rd	0010011	srli
0100000	shamt	rs1	101	rd	0010011	srai

I-format load:

		funct3	opcode		
imm[11:0]	rs1	000	rd	0000011	lb
imm[11:0]	rs1	001	rd	0000011	lh
imm[11:0]	rs1	010	rd	0000011	lw
imm[11:0]	rs1	100	rd	0000011	lbu
imm[11:0]	rs1	101	rd	0000011	lhu

S-format:

			funct3	opcode		
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw

B-format:

			funct3	opcode		
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	beq
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	bne
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	blt
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	bge
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	bltu
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	bgeu

J-format:

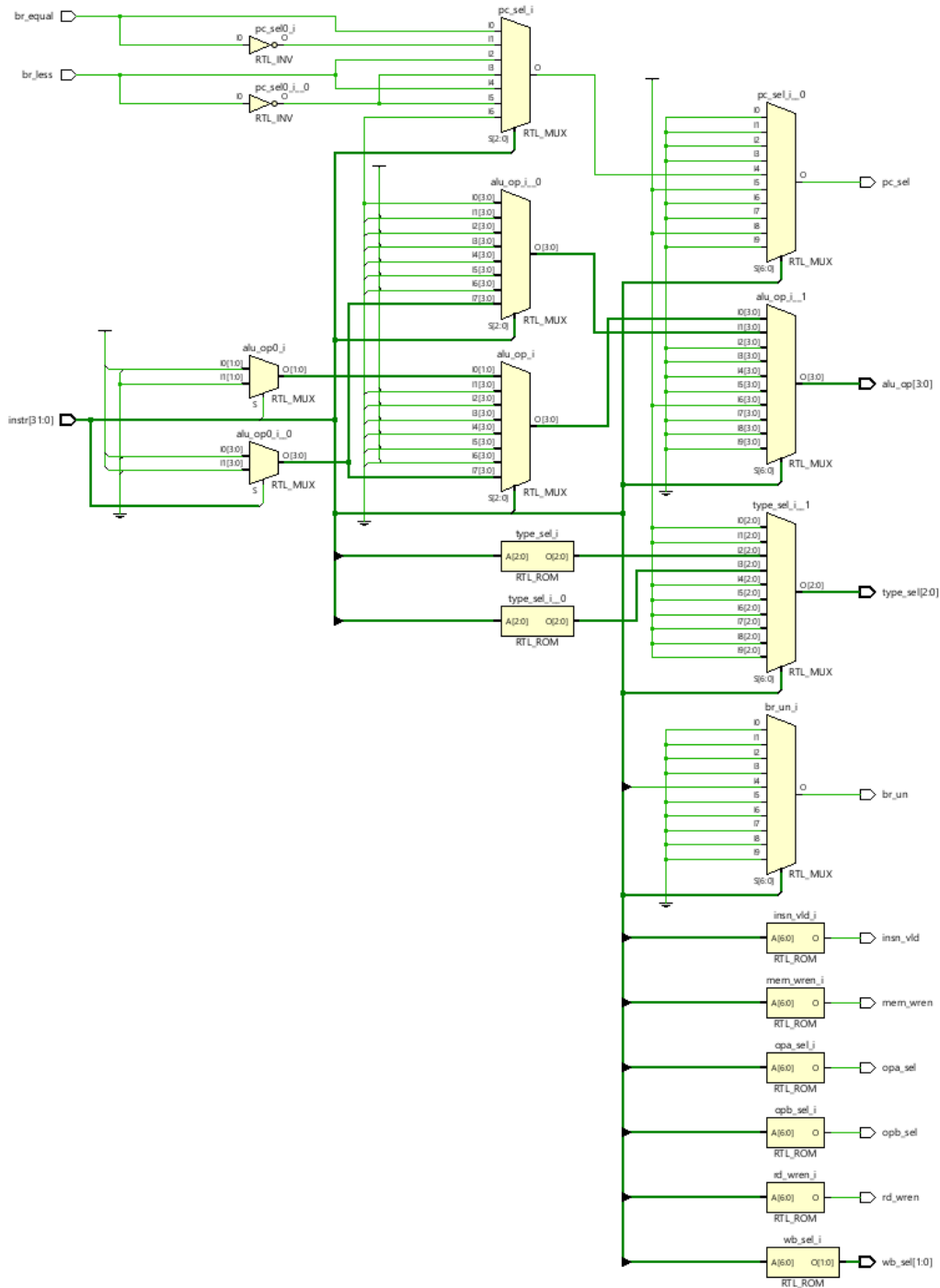
imm[20 10:1 11 19:12]	rd	1101111	JAL
imm[11:0]	rs1	000	rd 1100111 JALR

U-format:

imm[31:12]	rd	0110111	LUI
imm[31:12]	rd	0010111	AUIPC

- Thực hiện khối Control Unit bằng cách:
 - Đầu tiên xác định opcode để phân biệt các format lệnh khác nhau.
 - Sau đó, trong từng format sử dụng funct3 để xác định các lệnh tương ứng; nếu các lệnh có cùng funct3 thì tiếp tục xét trường funct7 để phân biệt.

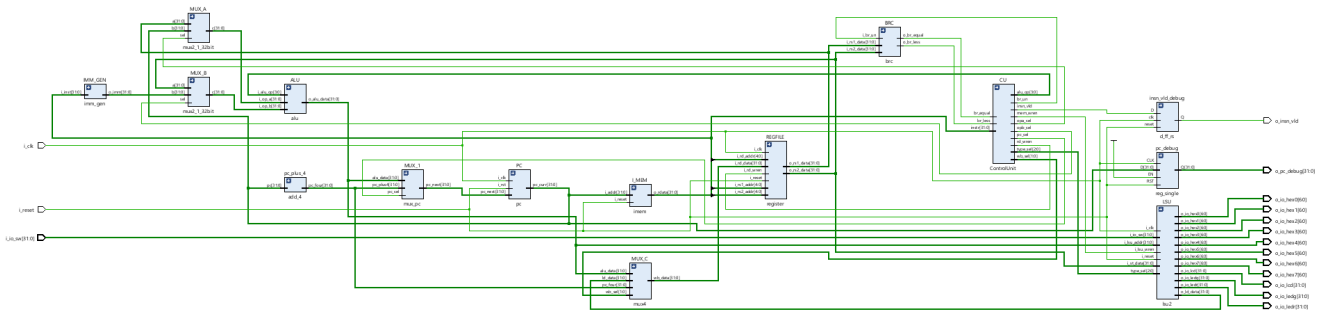
- Schematic:



Hình 3.1.16: CU schematic

3.1.7 Single cycle datapath

- Schematic:



Hình 3.1.17: SCDP

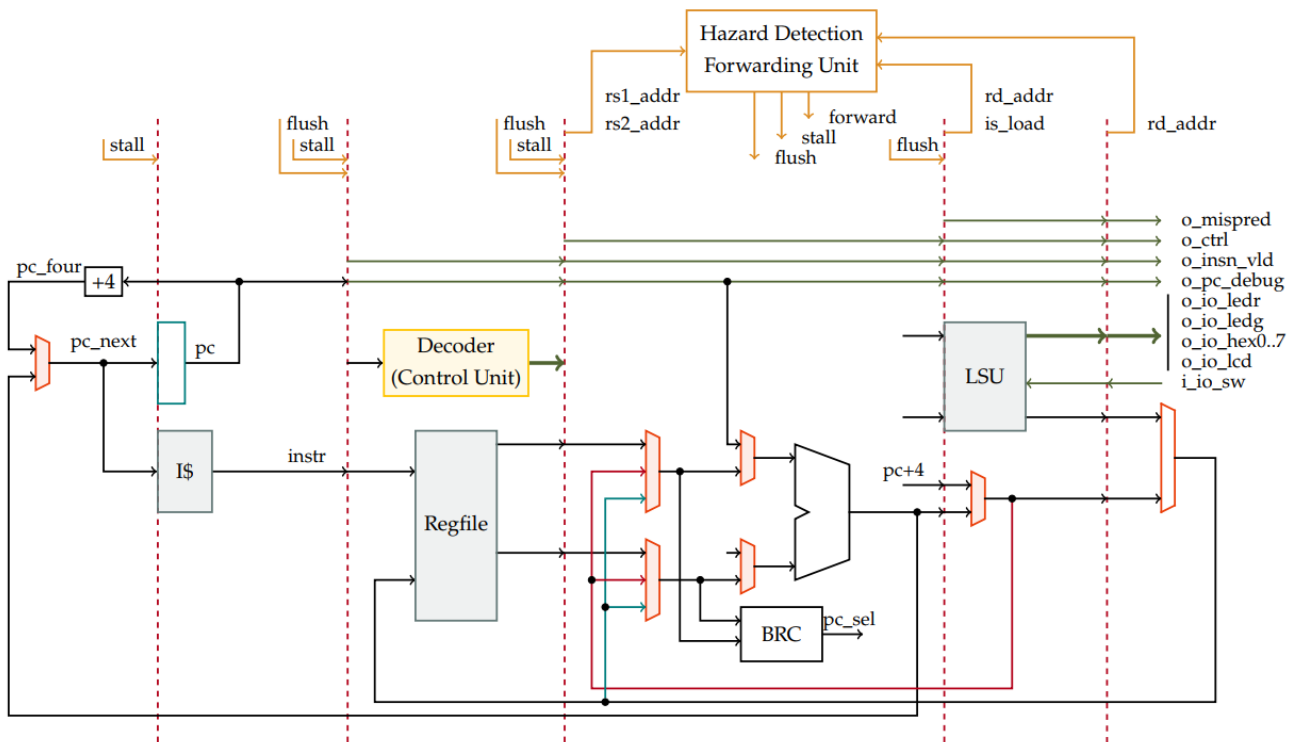
3.2 Chuyển đổi từ kiến trúc đơn chu kỳ sang kiến trúc pipeline

Ở phần trước, CPU RISC-V RV32I đã được trình bày theo kiến trúc đơn chu kỳ. Trong kiến trúc đơn chu kỳ, toàn bộ quá trình thực thi một lệnh, từ lấy lệnh, giải mã, thực thi, truy cập bộ nhớ đến ghi kết quả, đều được hoàn thành trong một chu kỳ clock. Cách tổ chức này có ưu điểm là đơn giản, dễ kiểm tra và phù hợp cho giai đoạn thiết kế ban đầu. Tuy nhiên, nhược điểm lớn của kiến trúc đơn chu kỳ là chu kỳ clock phải đủ dài để bao phủ đường truyền dữ liệu dài nhất của tất cả các loại lệnh. Do đó, tần số hoạt động tối đa của CPU bị giới hạn bởi đường tới hạn.

Để cải thiện hiệu năng, thiết kế được mở rộng sang kiến trúc pipeline 5 giai đoạn. Các giai đoạn chính của pipeline gồm:

- IF: Instruction Fetch – lấy lệnh từ bộ nhớ lệnh.
- ID: Instruction Decode – giải mã lệnh, đọc thanh ghi và sinh tín hiệu điều khiển.
- EX: Execute – thực hiện phép toán ALU, tính địa chỉ và xử lý điều kiện rẽ nhánh.
- MEM: Memory Access – truy cập bộ nhớ dữ liệu hoặc ngoại vi.
- WB: Write Back – ghi kết quả về Register File.

Việc chuyển từ kiến trúc đơn chu kỳ sang pipeline yêu cầu bổ sung các thanh ghi trung gian giữa các giai đoạn, đồng thời cần thêm các khối xử lý xung đột như Hazard Unit và Forwarding Unit để đảm bảo CPU thực thi đúng trong trường hợp các lệnh phụ thuộc dữ liệu hoặc thay đổi luồng chương trình.



Hình 3.2.1: CPU pipeline với khối phát hiện Hazard và khối Forwarding

3.2.1 Thanh ghi IF/ID

Thanh ghi IF/ID nằm giữa giai đoạn lấy lệnh và giai đoạn giải mã lệnh. Thanh ghi này có nhiệm vụ lưu trữ giá trị PC và lệnh vừa được đọc từ Instruction Memory. Ở chu kỳ tiếp theo, giai đoạn ID sẽ sử dụng các giá trị này để giải mã lệnh, đọc Register File và tạo các tín hiệu điều khiển.

Ngoài dữ liệu PC và instruction, thanh ghi IF/ID còn cần hỗ trợ các tín hiệu điều khiển như stall và flush. Tín hiệu stall được sử dụng khi pipeline cần tạm dừng để chờ dữ liệu sẵn sàng, còn tín hiệu flush được sử dụng để xóa lệnh đã nạp sai trong trường hợp branch hoặc jump làm thay đổi giá trị PC.

3.2.2 Thanh ghi ID/EX

Thanh ghi ID/EX nằm giữa giai đoạn giải mã và giai đoạn thực thi. Thanh ghi này lưu các dữ liệu đã đọc từ Register File, immediate đã được sinh bởi Immediate Generator, địa chỉ các thanh ghi rs1, rs2, rd, giá trị PC và các tín hiệu điều khiển cần thiết cho những giai đoạn sau.

Các tín hiệu điều khiển như chọn toán hạng ALU, mã điều khiển ALU, tín hiệu ghi thanh ghi, tín hiệu ghi bộ nhớ, tín hiệu chọn dữ liệu ghi về và tín hiệu nhận biết branch/jump được tạo ra ở giai đoạn ID rồi truyền qua thanh ghi ID/EX. Nhờ đó, giai đoạn EX có đủ thông tin để thực hiện phép toán tương ứng với lệnh hiện tại.

3.2.3 Thanh ghi EX/MEM

Thanh ghi EX/MEM nằm giữa giai đoạn thực thi và giai đoạn truy cập bộ nhớ. Thanh ghi này lưu kết quả tính toán từ ALU, dữ liệu rs2 dùng cho lệnh store, địa chỉ thanh ghi đích rd, giá trị PC + 4 và các tín hiệu điều khiển cần thiết cho giai đoạn MEM và WB.

Đối với lệnh load/store, kết quả ALU thường được dùng làm địa chỉ truy cập bộ nhớ. Đối với các lệnh tính toán số học hoặc logic, kết quả ALU sẽ được truyền tiếp đến giai đoạn WB để ghi

về Register File. Vì vậy, thanh ghi EX/MEM đóng vai trò quan trọng trong việc giữ đúng dữ liệu của từng lệnh khi các lệnh đang được xử lý chồng lấp trong pipeline.

3.2.4 Thanh ghi MEM/WB

Thanh ghi MEM/WB nằm giữa giai đoạn truy cập bộ nhớ và giai đoạn ghi kết quả. Thanh ghi này lưu dữ liệu đọc từ bộ nhớ đối với lệnh load, kết quả ALU đối với lệnh tính toán, giá trị $PC + 4$ đối với lệnh jump-and-link, địa chỉ thanh ghi đích và tín hiệu cho phép ghi thanh ghi.

Ở giai đoạn WB, dữ liệu từ thanh ghi MEM/WB được đưa vào bộ chọn Write Back để quyết định giá trị nào sẽ được ghi về Register File. Việc tách riêng giai đoạn MEM và WB giúp pipeline duy trì được luồng xử lý liên tục giữa các lệnh.

3.2.5 Control Unit trong kiến trúc pipeline

Trong kiến trúc pipeline, Control Unit được đặt chủ yếu ở giai đoạn ID. Khối này nhận đầu vào là lệnh 32-bit và thực hiện giải mã dựa trên các trường opcode, funct3 và funct7. Từ đó, Control Unit tạo ra các tín hiệu điều khiển tương ứng cho từng loại lệnh trong tập lệnh RV32I.

Bảng sau trình bày một số tín hiệu điều khiển chính của Control Unit.

Bảng 3.2.1: Đặc tả tín hiệu của khối Control Unit

Signal name	Width	Direction	Description
instr_ID	32	Input	Instruction 32 bit
br_un_ID	1	Output	Compare unsigned. Set to 1 for unsigned comparison.
rd_wren	1	Output	Register write enable. Set to 1 when writing data to Register File.
opa_sel_ID	1	Output	Operand A select. 0 selects <code>rs1</code> , 1 selects PC.
opb_sel_ID	1	Output	Operand B select. 0 selects <code>rs2</code> , 1 selects immediate.
alu_op_ID	4	Output	Select ALU operation.
mem_wren_ID	1	Output	Memory write enable. Set to 1 for store instructions.
type_sel_ID	3	Output	Branch or instruction type select.
wb_sel_ID	1	Output	Select data source for write back.
is_branch_ID	1	Output	Branch instruction flag. Set to 1 if the instruction is a branch.
is_jal_ID	1	Output	Jump and Link instruction flag. Set to 1 if the instruction is JAL.
is_jalr_ID	1	Output	Jump and Link Register instruction flag. Set to 1 if the instruction is JALR.
o_ctrl_ID	1	Output	Control flow instruction flag.
is_ld_ID	1	Output	Load instruction flag. Set to 1 if the instruction is a load.

3.2.6 Branch Unit

Branch Unit được đặt ở giai đoạn EX, có nhiệm vụ quyết định giá trị tiếp theo của PC. Đối với các lệnh thông thường, PC được cập nhật bằng $PC + 4$. Đối với các lệnh branch hoặc jump, PC có thể được cập nhật bằng địa chỉ đích do ALU tính toán.

Đối với lệnh branch, Branch Unit nhận các tín hiệu so sánh từ khối Branch Comparator như `br_less` và `br_equal`. Dựa vào loại lệnh branch được xác định bởi `type_sel`, khối này quyết định điều kiện rẽ nhánh có thỏa mãn hay không. Nếu điều kiện đúng, tín hiệu `pc_sel` được bật để chọn địa chỉ nhánh làm PC tiếp theo.

Đối với lệnh JAL và JALR, việc thay đổi PC là không điều kiện. Vì vậy, khi `is_jal` hoặc `is_jalr` được kích hoạt, Branch Unit sẽ tạo tín hiệu chọn PC mới tương ứng.

Bảng 3.2.2: Branch Unit Specification

Signal name	Width	Direction	Description
<code>type_sel_EX</code>	3	input	Branch type select signal.
<code>br_less_EX</code>	1	input	Less-than flag. Set to 1 if <code>rs1 < rs2</code> .
<code>br_equal_EX</code>	1	input	Equal flag. Set to 1 if <code>rs1 = rs2</code> .
<code>is_jal_EX</code>	1	input	Jump and Link instruction flag. Set to 1 if the instruction is JAL.
<code>is_jalr_EX</code>	1	input	Jump and Link Register instruction flag. Set to 1 if the instruction is JALR.
<code>is_branch_EX</code>	1	input	Branch enable control signal. Set to 1 if the instruction is a branch.
<code>pc_sel</code>	1	output	PC select signal. 0 selects $PC + 4$, 1 selects branch or jump target address.

3.2.7 Hazard Unit

Khi CPU hoạt động theo kiến trúc pipeline, nhiều lệnh được thực hiện đồng thời ở các giai đoạn khác nhau. Điều này làm xuất hiện các xung đột, hay còn gọi là hazard. Trong thiết kế này, Hazard Unit được sử dụng để phát hiện và xử lý các trường hợp có thể làm CPU thực thi sai.

Hai loại hazard chính được xét đến là data hazard và control hazard.

Data hazard xảy ra khi một lệnh cần sử dụng kết quả của lệnh trước đó nhưng kết quả này chưa được ghi về Register File. Hazard Unit kiểm tra địa chỉ thanh ghi nguồn `rs1` và `rs2` của lệnh ở giai đoạn ID với địa chỉ thanh ghi đích `rd` của các lệnh ở các giai đoạn EX, MEM và WB. Nếu phát hiện có phụ thuộc dữ liệu và không thể xử lý bằng forwarding, pipeline sẽ được tạm dừng bằng tín hiệu stall.

Control hazard xảy ra khi lệnh branch hoặc jump làm thay đổi luồng thực thi chương trình. Do CPU đã nạp sẵn các lệnh kế tiếp theo hướng $PC + 4$, nếu branch hoặc jump được thực hiện thì các lệnh đã nạp sai cần bị loại bỏ bằng tín hiệu flush.

Bảng 3.2.3: Hazard Unit Specification

Signal name	Width	Direction	Description
rs1_addr_ID	5	input	Address of the first source register.
rs2_addr_ID	5	input	Address of the second source register.
rd_x	5	input	Address of the destination register, with x representing EX, MEM or WB stage.
rd_wren_x	1	input	Write enable signal for the destination register, with x representing EX, MEM or WB stage. Set to 1 if writing.
pc_taken_EX	1	input	Branch taken signal from EX stage. Set to 1 if branch or jump is taken.
stall_PC	1	output	PC stall signal. Set to 1 when PC needs to be stalled.
stall_IF_ID	1	output	IF/ID pipeline register stall signal. Set to 1 when IF/ID needs to be stalled.
flush_IF_ID	1	output	IF/ID pipeline register flush signal. Set to 1 when IF/ID needs to be flushed.
flush_ID_EX	1	output	ID/EX pipeline register flush signal. Set to 1 when ID/EX needs to be flushed.

3.2.8 Forwarding Unit

Để giảm số chu kỳ stall, thiết kế sử dụng khối *Forwarding Unit*. Khối này có nhiệm vụ kiểm tra sự phụ thuộc dữ liệu giữa lệnh đang ở giai đoạn EX với các lệnh đang ở giai đoạn MEM và WB. Nếu phát hiện thanh ghi nguồn của lệnh hiện tại trùng với thanh ghi đích của lệnh trước đó, dữ liệu sẽ được chuyển tiếp trực tiếp từ tầng MEM hoặc WB về đầu vào ALU ở tầng EX.

Trong thiết kế này, Forwarding Unit nhận địa chỉ thanh ghi nguồn **rs1_addr_EX**, **rs2_addr_EX** của lệnh đang ở tầng EX. Đồng thời, khối này nhận địa chỉ thanh ghi đích **rd_MEM**, **rd_WB** và tín hiệu cho phép ghi thanh ghi **rd_wren_MEM**, **rd_wren_WB** từ các tầng sau. Nếu địa chỉ thanh ghi trùng nhau, thanh ghi đích khác **x0** và lệnh ở tầng sau có ghi về Register File, Forwarding Unit sẽ tạo tín hiệu điều khiển **forward_A** hoặc **forward_B** để chọn nguồn dữ liệu phù hợp.

Bảng 3.2.4: Forwarding Unit Specification

Signal name	Width	Direction	Description
rs1_addr_EX	5	input	Address of the first source register in EX stage.
rs2_addr_EX	5	input	Address of the second source register in EX stage.
rd_MEM	5	input	Address of the destination register in MEM stage.
rd_wren_MEM	1	input	Register write enable signal in MEM stage.
rd_WB	5	input	Address of the destination register in WB stage.
rd_wren_WB	1	input	Register write enable signal in WB stage.
forward_A	2	output	Select signal for the first ALU operand.
forward_B	2	output	Select signal for the second ALU operand.

Nguyên lý hoạt động của Forwarding Unit

Forwarding Unit sử dụng các bộ so sánh 5 bit để so sánh địa chỉ thanh ghi nguồn ở tầng EX với địa chỉ thanh ghi đích ở tầng MEM và WB. Cụ thể:

- So sánh rs1_addr_EX với rd_MEM và rd_WB.
- So sánh rs2_addr_EX với rd_MEM và rd_WB.

Nếu thanh ghi đích bằng x0, Forwarding Unit sẽ không thực hiện forwarding vì trong kiến trúc RISC-V, thanh ghi x0 luôn có giá trị bằng 0 và không bị thay đổi bởi lệnh ghi thanh ghi.

Trong trường hợp cùng lúc tồn tại khả năng forwarding từ cả MEM và WB, dữ liệu từ tầng MEM được ưu tiên hơn vì đây là kết quả mới hơn so với dữ liệu ở tầng WB.

Bảng 3.2.5: Mã điều khiển của tín hiệu forwarding

Giá trị	Nguồn dữ liệu được chọn
00	Lấy dữ liệu gốc từ thanh ghi ID/EX. Đây là trường hợp không xảy ra hazard.
10	Forward dữ liệu từ tầng MEM về tầng EX. Đây là trường hợp được ưu tiên cao nhất.
01	Forward dữ liệu từ tầng WB về tầng EX. Trường hợp này xảy ra khi không có forwarding từ MEM nhưng có phụ thuộc với WB.

Kết nối Forwarding Unit trong tầng EX

Trong tầng EX, hai tín hiệu **forward_A** và **forward_B** được đưa vào hai bộ chọn dữ liệu 3 ngõ vào. Hai bộ chọn này quyết định dữ liệu thực sự được đưa vào ALU và Branch Comparator.

- Nếu `forward_A = 00`, toán hạng A lấy từ dữ liệu `rs1` đã lưu trong thanh ghi ID/EX.
- Nếu `forward_A = 10`, toán hạng A lấy từ kết quả ALU ở tầng MEM.
- Nếu `forward_A = 01`, toán hạng A lấy từ dữ liệu ghi về ở tầng WB.
- Tín hiệu `forward_B` hoạt động tương tự cho toán hạng B.

Sau khi qua bộ chọn forwarding, dữ liệu `src_a_fwd` và `src_b_fwd` được đưa đến các MUX chọn toán hạng ALU. Đồng thời, hai dữ liệu này cũng được đưa vào Branch Comparator để đảm bảo các lệnh branch sử dụng dữ liệu mới nhất.

Đối với lệnh store, dữ liệu `rs2` dùng để ghi xuống bộ nhớ cũng phải là dữ liệu đã được forwarding. Vì vậy, dữ liệu `src_b_fwd` được truyền sang thanh ghi EX/MEM thay vì dùng trực tiếp giá trị `rs2` cũ.

Chương 4

HIỆN THỰC CPU PIPELINE TRÊN KIT PYNQ-Z2

4.1 Mục tiêu hiện thực trên FPGA

Sau khi thiết kế và mô phỏng CPU RISC-V RV32I pipeline ở mức RTL, thiết kế được hiện thực trên kit FPGA PYNQ-Z2 nhằm kiểm chứng hoạt động trên phần cứng thực tế. Mục tiêu của chương này là trình bày quá trình xây dựng module top, ánh xạ các tín hiệu vào/ra của CPU với tài nguyên vật lý trên kit, xây dựng file ràng buộc chân .xdc và kiểm chứng hoạt động của CPU thông qua chương trình điều khiển LED bằng button.

Trong thiết kế này, CPU pipeline nhận tín hiệu clock từ kit PYNQ-Z2, nhận tín hiệu điều khiển từ các button và xuất kết quả ra các LED trên board. Các button được đưa vào CPU thông qua ngõ vào ngoại vi `i_io_sw`, trong khi LED được điều khiển bởi ngõ ra `o_io_ledr`. Việc giao tiếp giữa chương trình RISC-V và ngoại vi được thực hiện theo cơ chế memory-mapped I/O.

4.2 Tổng quan module top

Module top được sử dụng trong quá trình hiện thực trên PYNQ-Z2 là `top_pynq_pipeline`. Module này đóng vai trò bao bọc CPU pipeline và kết nối các tín hiệu của CPU với các chân vật lý của FPGA.

Các cổng chính của module top gồm:

- `sysclk`: xung clock hệ thống từ kit PYNQ-Z2.
- `btn[3:0]`: bốn nút nhấn trên kit, trong đó `btn[0]` được dùng làm reset.
- `led[3:0]`: bốn LED vật lý trên kit, dùng để hiển thị kết quả từ CPU.

Bảng 4.2.1 trình bày đặc tả các cổng vào/ra của module top.

Bảng 4.2.1: Đặc tả cổng vào/ra của module `top_pynq_pipeline`

Tên tín hiệu	Độ rộng	Hướng	Chức năng
<code>sysclk</code>	1 bit	Input	Clock hệ thống 125 MHz từ kit PYNQ-Z2.
<code>btn[3:0]</code>	4 bit	Input	Các nút nhấn trên board. <code>btn[0]</code> dùng làm reset, <code>btn[3:1]</code> dùng làm ngõ vào cho CPU.
<code>led[3:0]</code>	4 bit	Output	LED vật lý trên kit, được điều khiển bởi <code>o_io_ledr[3:0]</code> của CPU.

Bên trong module top, CPU pipeline được khởi tạo thông qua module `pipelined`. Các tín hiệu debug như `o_pc_debug`, `o_insn_vld`, `o_ctrl` và `o_mispred` vẫn được giữ lại để phục vụ quá trình kiểm tra, mặc dù trong thiết kế hiện thực trên LED chỉ sử dụng ngõ ra `o_io_ledr`.

4.3 Thiết kế bộ chia clock

Kit PYNQ-Z2 cung cấp xung clock hệ thống tại chân `sysclk` với tần số 125 MHz. Nếu đưa trực tiếp xung này vào CPU, chương trình điều khiển LED có thể chạy quá nhanh, gây khó khăn khi quan sát bằng mắt thường. Vì vậy, thiết kế sử dụng một bộ đếm để chia clock trước khi cấp cho CPU.

Trong module top, thanh ghi `div_cnt` được tăng lên tại mỗi cạnh lên của `sysclk`. Tín hiệu `cpu_clk_raw` được lấy từ bit thứ 7 của bộ đếm:

```
always_ff @(posedge sysclk) begin
    div_cnt <= div_cnt + 1'b1;
end
```

```
assign cpu_clk_raw = div_cnt[7];
```

Với clock đầu vào 125 MHz, tần số của `div_cnt[7]` xấp xỉ:

$$f_{cpu} = \frac{125 \text{ MHz}}{2^8} \approx 488.28 \text{ kHz}$$

Sau đó, tín hiệu clock đã chia được đưa qua BUFG để tạo clock toàn cục ổn định cho FPGA:

```
BUFG bufg_cpu_clk (
    .I(cpu_clk_raw),
    .O(cpu_clk)
);
```

Việc sử dụng BUFG giúp tín hiệu clock được phân phối qua mạng clock chuyên dụng của FPGA, hạn chế skew và đảm bảo thiết kế hoạt động ổn định hơn so với việc dùng tín hiệu logic thông thường làm clock.

4.4 Thiết kế reset

Tín hiệu reset của CPU là tín hiệu tích cực mức thấp. Trong module top, `btn[0]` được dùng làm nút reset. Khi nhấn BTN0, tín hiệu reset đưa vào CPU sẽ ở mức thấp, làm CPU quay về trạng thái ban đầu.

Để đồng bộ reset với clock của CPU, thiết kế sử dụng hai thanh ghi trung gian `rst_n_meta` và `rst_n_cpu`:

```
always_ff @(posedge cpu_clk) begin
    rst_n_meta <= ~btn[0];
    rst_n_cpu  <= rst_n_meta;
end
```

Trong đó:

- Khi `btn[0] = 0`, tín hiệu `rst_n_cpu = 1`, CPU hoạt động bình thường.
- Khi `btn[0] = 1`, tín hiệu `rst_n_cpu = 0`, CPU được reset.

Việc sử dụng hai tầng đồng bộ giúp giảm nguy cơ metastability khi tín hiệu button không đồng bộ với clock nội bộ của CPU.

4.5 Đồng bộ tín hiệu button vào CPU

Ba button còn lại, gồm `btn[1]`, `btn[2]` và `btn[3]`, được dùng làm tín hiệu ngõ vào cho chương trình chạy trên CPU. Các tín hiệu này được đưa qua một thanh ghi đồng bộ theo `cpu_clk`:

```
logic [2:0] btn_cpu;

always_ff @(posedge cpu_clk) begin
    btn_cpu <= btn[3:1];
end
```

Sau đó, ba bit button được mở rộng thành tín hiệu 32 bit `i_io_sw`:

```
assign i_io_sw = {29'b0, btn_cpu};
```

Cách ánh xạ này cho phép chương trình RISC-V đọc trạng thái button thông qua vùng địa chỉ ngoại vi. Trong đó:

- `i_io_sw[0]` nhận giá trị từ `btn[1]`.
- `i_io_sw[1]` nhận giá trị từ `btn[2]`.
- `i_io_sw[2]` nhận giá trị từ `btn[3]`.

4.6 Kết nối CPU pipeline với ngoại vi

Module `pipelined` có nhiều cổng ngoại vi như LED đỏ, LED xanh, LED 7 đoạn, LCD và switch. Tuy nhiên, trong phạm vi kiểm chứng trên kit PYNQ-Z2, thiết kế chỉ sử dụng các button làm ngõ vào và bốn LED vật lý làm ngõ ra.

Tín hiệu LED vật lý được lấy từ bốn bit thấp của `o_io_ledr`:

```
assign led = o_io_ledr[3:0];
```

Bảng 4.6.1 trình bày ánh xạ tín hiệu giữa CPU và kit PYNQ-Z2.

Bảng 4.6.1: Ánh xạ tín hiệu ngoại vi trong module top

Tài nguyên trên kit	Tín hiệu trong CPU	Chức năng
BTN0	<code>i_reset</code>	Reset CPU, tác động mức thấp sau khi đảo tín hiệu.
BTN1	<code>i_io_sw[0]</code>	Ngõ vào điều khiển chế độ LED thứ nhất.
BTN2	<code>i_io_sw[1]</code>	Ngõ vào điều khiển chế độ LED thứ hai.
BTN3	<code>i_io_sw[2]</code>	Ngõ vào điều khiển chế độ LED thứ ba.
LED0	<code>o_io_ledr[0]</code>	Hiển thị bit 0 của dữ liệu LED.
LED1	<code>o_io_ledr[1]</code>	Hiển thị bit 1 của dữ liệu LED.
LED2	<code>o_io_ledr[2]</code>	Hiển thị bit 2 của dữ liệu LED.
LED3	<code>o_io_ledr[3]</code>	Hiển thị bit 3 của dữ liệu LED.

4.7 File ràng buộc chân XDC

Để Vivado biết các cổng logic trong module top được nối với chân vật lý nào trên FPGA, thiết kế cần file ràng buộc chân .xdc. File `pynq_z2_pipeline.xdc` khai báo chân cho clock, button và LED.

4.7.1 Ràng buộc clock

Clock hệ thống của PYNQ-Z2 được nối với chân H16. Tần số clock là 125 MHz, tương ứng chu kỳ 8 ns:

```
set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { sysclk }]
create_clock -add -name sys_clk_pin -period 8.000 -waveform {0 4} [get_ports { sysclk }]
```

Khai báo `create_clock` giúp Vivado thực hiện phân tích timing đúng với tần số clock đầu vào của thiết kế.

4.7.2 Ràng buộc button

Bốn button trên kit được ánh xạ đến các chân vật lý như Bảng 4.7.1.

Bảng 4.7.1: Ràng buộc chân cho button trên PYNQ-Z2

Tín hiệu	Chân FPGA	Chức năng
<code>btn[0]</code>	D19	Reset CPU.
<code>btn[1]</code>	D20	Ngõ vào <code>i_io_sw[0]</code> .
<code>btn[2]</code>	L20	Ngõ vào <code>i_io_sw[1]</code> .
<code>btn[3]</code>	L19	Ngõ vào <code>i_io_sw[2]</code> .

4.7.3 Ràng buộc LED

Bốn LED vật lý trên kit được ánh xạ đến các chân FPGA như Bảng 4.7.2.

Bảng 4.7.2: Ràng buộc chân cho LED trên PYNQ-Z2

Tín hiệu	Chân FPGA	Chức năng
<code>led[0]</code>	R14	Hiển thị <code>o_io_ledr[0]</code> .
<code>led[1]</code>	P14	Hiển thị <code>o_io_ledr[1]</code> .
<code>led[2]</code>	N16	Hiển thị <code>o_io_ledr[2]</code> .
<code>led[3]</code>	M14	Hiển thị <code>o_io_ledr[3]</code> .

Tất cả các tín hiệu I/O đều sử dụng chuẩn điện áp LVCMOS33, phù hợp với tài nguyên I/O của kit PYNQ-Z2.

4.8 Chương trình kiểm thử LED

Chương trình kiểm thử được viết bằng assembly RISC-V, sau đó được biên dịch thành mã máy dạng file `.hex`. File này được nạp vào Instruction Memory bằng lệnh `$readmemh`. Khi CPU

hoạt động, chương trình sẽ đọc trạng thái button từ địa chỉ 0x10010000 và ghi dữ liệu điều khiển LED ra địa chỉ 0x10000000.

Trong chương trình, thanh ghi x8 được dùng để lưu địa chỉ cơ sở của LED, còn thanh ghi x9 được dùng để lưu địa chỉ cơ sở của button/switch.

```
lui x8, 0x10000      # x8 = 0x10000000, LED base address
lui x9, 0x10010      # x9 = 0x10010000, switch/button base address
```

Bảng 4.8.1: Các chế độ hoạt động của chương trình kiểm thử LED

Button	Bit kiểm tra	Chức năng
BTN1	i_io_sw[0]	LED chạy theo thứ tự 0001 -> 0010 -> 0100 -> 1000.
BTN2	i_io_sw[1]	LED chạy theo thứ tự 1000 -> 0100 -> 0010 -> 0001.
BTN3	i_io_sw[2]	LED sáng toàn bộ 1111, sau đó tắt 0000.

4.9 Kết quả hiện thực trên kit PYNQ-Z2

Sau khi mô phỏng đúng chức năng, thiết kế được tổng hợp, implementation và generate bitstream trên Vivado. Bitstream sau đó được nạp xuống kit PYNQ-Z2 để kiểm chứng. Kết quả kiểm chứng được đánh giá thông qua phản ứng của LED khi nhấn các button.

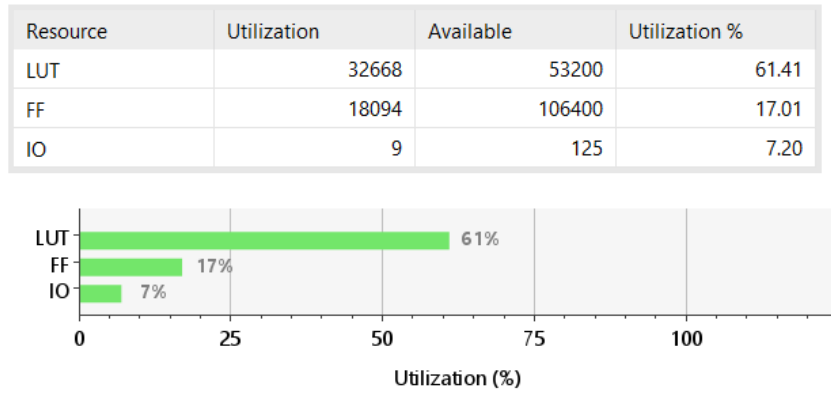
Bảng 4.9.1: Kết quả kiểm chứng chương trình LED trên PYNQ-Z2

Trường hợp kiểm thử	Kết quả mong đợi	Đánh giá
Nhấn BTN0	CPU reset, LED trở về trạng thái ban đầu.	Đạt
Nhấn BTN1	LED chạy theo thứ tự 0001 -> 0010 -> 0100 -> 1000.	Đạt
Nhấn BTN2	LED chạy theo thứ tự 1000 -> 0100 -> 0010 -> 0001.	Đạt
Nhấn BTN3	Bốn LED sáng toàn bộ rồi tắt.	Đạt

Video kết quả: <https://drive.google.com/file/d/1GpDVK3qwtfwTmZq-ffx0M9re3Bt4kUdS/view?usp=sharing>

4.10 Kết quả synthesis

Sau bước synthesis, Vivado tạo báo cáo tài nguyên và timing cho thiết kế.



Hình 4.10.1: Kết quả sử dụng tài nguyên FPGA của thiết kế

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	Bonded IOB (125)	BUFGCTRL (32)
top_pynq_pipeline	32668	18094	4394	2112	8913	32668	9	2
your_cpu (pipelined)	32666	18081	4394	2112	8908	32666	0	0
IMEM (imem)	89	0	23	0	27	89	0	0
LSU (lsu2)	8896	16539	4352	2112	7974	8896	0	0
REG_EX_MEM (EX_MEM)	22224	179	0	0	6155	22224	0	0
REG_FILE (register)	705	992	0	0	399	705	0	0
REG_ID_EX (ID_EX)	490	161	19	0	224	490	0	0
REG_IF_ID (IF_ID)	32	67	0	0	40	32	0	0
REG_MEM_WB (MEM_WB)	34	111	0	0	73	34	0	0

Hình 4.10.2: Kết quả sử dụng tài nguyên theo từng module trong thiết kế

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.057 ns	Worst Hold Slack (WHS): 0.252 ns	Worst Pulse Width Slack (WPWS): 3.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 8	Total Number of Endpoints: 8	Total Number of Endpoints: 9

All user specified timing constraints are met.

Hình 4.10.3: Kết quả phân tích timing của thiết kế

Dựa trên kết quả sử dụng tài nguyên tổng quát, thiết kế sử dụng 32668 LUT trên tổng số 53200 LUT, tương ứng 61.41%, và 18094 FF trên tổng số 106400 FF, tương ứng 17.01%. Số lượng IO sử dụng là 9 trên tổng số 125 chân, tương ứng 7.20%. Kết quả này cho thấy thiết kế chiếm tài nguyên LUT tương đối lớn, trong khi FF và IO vẫn còn dư nhiều.

Ở kết quả sử dụng tài nguyên theo từng module, phần lớn tài nguyên nằm trong module CPU chính `your_cpu`. Trong đó, các khối chiếm nhiều tài nguyên nhất là `REG_EX_MEM` và `LSU`. Điều này cho thấy các thanh ghi pipeline và khối xử lý load/store, ngoại vi có ảnh hưởng lớn đến lượng tài nguyên sử dụng.

Kết quả timing cho thấy thiết kế thỏa mãn các ràng buộc thời gian. Giá trị WNS đạt 6.057 ns, TNS bằng 0.000 ns và số endpoint lỗi bằng 0. Hold timing và pulse width timing cũng không có vi phạm. Do đó, thiết kế có thể được xem là đạt timing sau implementation.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Trong đồ án này, CPU RISC-V RV32I đã được thiết kế bằng ngôn ngữ SystemVerilog. Thiết kế ban đầu được xây dựng dựa trên các khối chức năng cơ bản như Program Counter, Instruction Memory, Register File, Immediate Generator, ALU, Branch Comparator, Load Store Unit và Control Unit. Từ nền tảng đó, thiết kế được mở rộng sang kiến trúc pipeline 5 giai đoạn gồm IF, ID, EX, MEM và WB.

CPU pipeline được bổ sung các thanh ghi pipeline IF/ID, ID/EX, EX/MEM và MEM/WB để truyền dữ liệu và tín hiệu điều khiển giữa các giai đoạn. Ngoài ra, thiết kế tích hợp Hazard Unit để xử lý load-use hazard và control hazard, đồng thời sử dụng Forwarding Unit để giảm số chu kỳ stall do data hazard.

Thiết kế được hiện thực trên kit PYNQ-Z2 thông qua module `top_pynq_pipeline`. Các button trên kit được dùng làm ngõ vào điều khiển, còn bốn LED được dùng để quan sát kết quả thực thi chương trình. Chương trình kiểm thử LED được biên dịch thành file `.hex` và nạp vào Instruction Memory của CPU.

5.2 Hạn chế

Thiết kế hiện tại mới tập trung vào tập lệnh RV32I cơ bản và các ngoại vi đơn giản như button và LED. Một số kỹ thuật nâng cao như branch prediction, cache, interrupt, UART hoặc giao tiếp AXI chưa được triển khai đầy đủ trong phạm vi đồ án. Ngoài ra, chương trình kiểm thử trên kit còn đơn giản, chủ yếu nhằm kiểm chứng khả năng đọc ngõ vào và điều khiển ngõ ra.

Bên cạnh đó, clock dùng cho CPU được tạo bằng cách lấy một bit từ bộ đếm chia clock. Cách làm này đơn giản và phù hợp cho demo LED, nhưng trong các thiết kế hoàn chỉnh hơn nên sử dụng Clocking Wizard hoặc PLL/MMCM để tạo clock có chất lượng tốt hơn và dễ kiểm soát timing hơn.

5.3 Hướng phát triển

Trong tương lai, thiết kế có thể được phát triển theo các hướng sau:

- Bổ sung bộ dự đoán nhánh để giảm số lần flush do control hazard.
- Mở rộng chương trình kiểm thử để bao phủ đầy đủ hơn tập lệnh RV32I.
- Tối ưu tài nguyên phần cứng và cải thiện tần số hoạt động tối đa.
- Bổ sung UART để quan sát dữ liệu từ CPU dễ dàng hơn.

- Tích hợp thêm LCD, switch hoặc các ngoại vi khác trên kit FPGA.
- Nghiên cứu kết nối CPU với bus AXI để mở rộng khả năng giao tiếp với các IP khác trong hệ thống FPGA.
- Cải tiến bộ nhớ lệnh và bộ nhớ dữ liệu theo hướng sử dụng Block RAM để giảm tài nguyên logic.

Tài liệu tham khảo

- [1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, 2019. Available: <https://riscv.org/technical/specifications/>
- [2] David A. Patterson and John L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, Morgan Kaufmann.
- [3] Meghraj, *RISC-V CPU from Scratch*, GitHub repository. Available: <https://github.com/Git-Meghraj/riscv-cpu-from-scratch>
- [4] UC Berkeley CS61C, *Great Ideas in Computer Architecture: RISC-V Datapath, Control and Pipeline Lecture Slides*, Course lecture slides, 2022.
- [5] TUL Corporation, *PYNQ-Z2 Board Reference Manual*
- [6] Tran Chi Nhan, *Computer Architecture – Milestone 2 – Single Cycle RISC-V*, GitHub repository. Available: https://github.com/tnhan114/Computer_Architecture---Milestone_2---Single_Cycle_RISCV
- [7] Tran Chi Nhan, *Computer Architecture – Milestone 3 – Pipelined RISC-V*, GitHub repository. Available: https://github.com/tnhan114/Computer_Architecture---Milestone_3---Pipelined_RISCV